

- Linee guida e standard -

Linee Guida per l'uso di SPRINGLINE2

Codice Documento: **ARIA-W8B1-LGD@91**

Revisione del Documento: **1.18**

Data revisione: **16/10/2025**



Cronologia delle Revisioni

Revisione	Data	Sintesi delle Modifiche
1.0	11/12/2023	Prima emissione
1.1	24/01/2024	Gestione del JWT anche nel cookie Nuovo modulo springline2-batch
1.2	07/02/2024	Aggiunta Appendice B per l'utilizzo degli archetype Aggiunta Appendice C con i valori di default delle properties
1.3	09/02/2024	In appendice B corretto groupid degli archetype
1.4	08/03/2024	Aggiunta properties server.tomcat.max-http-form-post-size=2MB Aggiornato elenco versioni disponibili
1.5	15/04/2024	Capitolo 5.4.2 Client Rest: aggiunta gestione retry Capitolo 5.6 Handening Tomcat Embedded: evidenziato il context-path e il fatto che non deve essere modificato Aggiunta in appendice C le nuove properties spl.client-rest.retry.* Aggiornato elenco versioni disponibili
1.6	29/05/2024	Capitolo 5.2.3 Auditlog descritte le properties valorizzate in caso di chiamata http Aggiunto Capitolo 9.1.3 Auditlog con le properties valorizzate in caso di messaggio
1.7	30/05/2024	Capitolo 5.2 5.2 SpringLine 2 – Logging specificate nuove funzionali delle versioni 2024.05.01 e 2023.06.01 Aggiunto capitolo 5.5.7 StrictHttpFirewall Aggiunta in appendice C le nuove properties spl.security.http-firewall.* Aggiunta appendice D con il CHANGELOG di SPRINGLINE2
1.8	23/07/2024	Aggiornata l'appendice A Modificato in appendice C il valore di default della proprietà spl.http.logging.app.excluded-content-types Aggiornato appendice D con il CHANGELOG di SPRINGLINE2
1.9	30/08/2024	Aggiornata l'appendice A Aggiornato appendice D con il CHANGELOG di SPRINGLINE2
1.10	24/09/2024	Aggiornata l'appendice A Aggiornato appendice D con il CHANGELOG di SPRINGLINE2
1.11	18/11/2024	Capitolo 5.2 – indicati i file di configurazione di logback presenti Aggiunto capitolo 5.2.4 Cifratura dei log Riscritto completamente il capitolo 5.3 Capitolo 5.5.5.3 API Manager: aggiunte in tabelle le nuove proprietà di configurazione Modifiche in vari capitoli a fronte del nuovo modulo springline2-config Aggiornata l'appendice A Aggiornata l'appendice C Aggiornato appendice D con il CHANGELOG di SPRINGLINE2 Aggiunta appendice E con i file di configurazione di logback
1.12	24/02/2025	Capitolo 5.3 Springline2 – Config per nuova gestione cache della configurazione Aggiornata l'appendice A Aggiornata l'appendice C Aggiornato appendice D con il CHANGELOG di SPRINGLINE2
1.13	27/02/2025	Aggiornata l'appendice A Aggiornato appendice D con il CHANGELOG di SPRINGLINE2
1.14	28/02/2025	Aggiornata l'appendice A Aggiornato appendice D con il CHANGELOG di SPRINGLINE2

Revisione	Data	Sintesi delle Modifiche
1.15	09/05/2025	Capitolo 5.1 – Nuova gestione mappa di attributi applicativi Capitolo 5.3 - Specificata la corrispondenza tra key/value store di Consul e configurazione applicativa Aggiornato capitolo 9. SpringLine 2 - JMS per il passaggio da ActiveMQ Classic a ActiveMQ Artemis. Aggiornata Appendice A Aggiornata Appendice D con il CHANGELOG di SPRINGLINE2 Aggiornata Appendice E
1.16	23/05/2025	Capitolo 5.3 – Specificata la dipendenza da inserire nel pom.xml per l'utilizzo delle funzionalità di configurazione
1.17	08/09/2025	Aggiornato capitolo 9 per nuova proprietà Aggiornato tabella in capitolo 5.2.2.11 per nuova proprietà Aggiornata l'appendice A a seguito di nuove versioni di SPRINGLINE2 Aggiornata l'appendice C per l'aggiunta di alcune properties
1.18	16/10/2025	Modificato "Appendice B - Utilizzo degli archetype Maven" per eliminare un refuso

Limiti di utilizzo del documento

In base alla classificazione del documento. **Le modifiche rispetto alla versione precedente del documento sono evidenziate in giallo.**

Indice

1.	Introduzione	6
2.	Obiettivo	7
3.	Diagramma delle componenti	8
4.	Dipendenze Applicative	9
5.	SpringLine 2 – Core	11
5.1	SpringLine 2 – Context	12
5.1.1	Accesso al Context	15
5.2	SpringLine 2 – Logging	16
5.2.1	MON (Log di monitoraggio)	17
5.2.1.1	HTTP - IN	17
5.2.1.2	HTTP - OUT	18
5.2.1.3	Client Rest - IN	19
5.2.1.4	Client Rest - OUT	19
5.2.1.5	JMS LISTENER - IN	20
5.2.1.6	JMS LISTENER - OUT	21
5.2.1.7	JMS Client - IN	22
5.2.1.8	JMS Client - OUT	22
5.2.1.9	WS - IN	23
5.2.1.10	WS - OUT	24
5.2.1.11	HTTP	25
5.2.1.12	JMS - LISTENER	25
5.2.1.13	Client Rest	25
5.2.1.14	JMS Client	26
5.2.1.15	WS Client	26
5.2.2	MON-APP (Log di monitoraggio applicativo)	26
5.2.2.1	HTTP - IN	27
5.2.2.2	HTTP - OUT	27
5.2.2.3	Client Rest - IN	27
5.2.2.4	Client Rest - OUT	28
5.2.2.5	JMS LISTENER - IN	28
5.2.2.6	JMS LISTENER - OUT	28
5.2.2.7	JMS Client - IN	29
5.2.2.8	JMS Client - OUT	29
5.2.2.9	WS - IN	29
5.2.2.10	WS - OUT	30
5.2.2.11	HTTP	30
5.2.2.12	JMS - LISTENER	31
5.2.2.13	Client Rest	31
5.2.2.14	JMS Client	32
5.2.2.15	WS Client	32
5.2.3	AuditLog	33
5.2.3.1	Start/ShutDown Application	34
5.2.3.2	Utente non Autorizzato	35
5.2.3.3	JWT – Utilizzo Token di autorizzazione	36
5.2.3.4	Eccezione inaspettata	36
5.2.3.5	Eccezione validazione dati	37
5.2.4	Cifratura dei log	37
5.3	SpringLine 2 – Config	38
5.4	SpringLine 2 – API Rest	45
5.4.1	Provider Rest	45
5.4.2	Client Rest	46
5.4.3	MON	48
5.4.4	MON-APP	48
5.4.5	Propagation SpringLine 2	49
5.4.6	Propagation SpringLine	49
5.5	SpringLine 2 – Security	51
5.5.1	Configurazione Base SpringLine 2 - Security	52
5.5.2	Configurazione Custom SpringLine 2 - Security	53

5.5.3	Properties User.....	54
5.5.3.1	Anonymous	54
5.5.3.2	IAM	54
5.5.3.3	API Manager	55
5.5.3.4	Utente Applicativo (Cittadino/Operatore)	55
5.5.3.5	Servizio	57
5.5.3.6	Propagator.....	58
5.5.3.7	IDPC	58
5.5.3.8	SAMWEB	60
5.5.3.9	SpringLine	60
5.5.3.10	JWT	61
5.5.4	Authorities	62
5.5.5	Configurazioni applicative YAML/Properties	63
5.5.5.1	Anonymous	63
5.5.5.2	IAM	63
5.5.5.3	API Manager	64
5.5.5.4	Propagator.....	64
5.5.5.5	IDPC	64
5.5.5.6	SAMWEB	66
5.5.5.7	JWT	67
5.5.5.8	SpringLine	67
5.5.6	Accesso alle informazioni dell'Utente	68
5.5.6.1	Accesso mediante Contesto	68
5.5.6.2	Accesso mediante Annotations.....	69
5.5.6.2.1	Annotation User.....	69
5.5.6.2.2	Annotation Properties User	69
5.5.7	StrictHttpFirewall.....	71
5.6	Hardening Tomcat Embedded	72
6.	SpringLine 2 - WS.....	72
6.1	Configurazioni.....	73
6.1.1	Logging MON.....	73
6.1.2	Logging MON-APP	73
6.1.3	Propagazione Siss	74
6.2	Esempi di codice - WS	74
6.2.1	Configurazione	74
6.2.2	Client SOAP	75
7.	SpringLine 2 - OpenApi.....	76
7.1.1	EndPoint.....	76
7.1.2	Configurazione	77
7.1.3	Esempi.....	78
8.	SpringLine 2 – Data	80
8.1.1	Configurazione Data Source Yaml	80
8.1.2	Configurazione Properties.....	81
9.	SpringLine 2 – JMS	82
9.1.1	Configurazione JMS Listener	83
9.1.2	Configurazione JMS Client	83
9.1.3	Auditlog.....	84
10.	SpringLine 2 – BATCH	85
11.	Conclusione.....	86
12.	Appendice A – Versioni	87
13.	Appendice B - Utilizzo degli archetype Maven.....	90
14.	Appendice C – Default Application Properties	91
15.	Appendice D – CHANGELOG	96
16.	Appendice E – Configurazioni logback	99
16.1	logback-springline2.xml.....	99
16.2	logback-springline2-ocp.xml	100
16.3	logback-springline2-encrypt.xml.....	101
16.4	logback-springline2-ocp-encrypt.xml	102

1. Introduzione

SpringLine 2 va oltre una semplice evoluzione della sua versione precedente. La libreria è stata ridisegnata e modularizzata passando da uno Starter SpringBoot a una collezione di librerie indipendenti ed integrabili come moduli a sé stanti. Offre agli sviluppatori una serie di vantaggi:

Modularità e Flessibilità: gli sviluppatori hanno la libertà di scegliere e integrare solo le componenti necessarie, evitando di sovraccaricare l'applicazione con moduli superflui.

Maggiore Controllo sulle Configurazioni: La struttura a librerie permette un controllo più granulare sulle configurazioni, facilitando l'ottimizzazione e la personalizzazione dell'applicazione;

Aggiornamenti e Manutenzione Semplificati: grazie alla sua struttura modulare, con SpringLine 2 è possibile aggiornare ed evolvere la versione di ogni singolo modulo indipendentemente dagli altri. Questo approccio non solo garantisce maggiore stabilità, ma riduce anche i potenziali problemi di compatibilità che potrebbero emergere quando una dipendenza di uno specifico modulo viene aggiornata;

Integrazione con Altri Starter: la flessibilità offerta permette di combinare le funzionalità di Spring Line 2 con altri starter di Spring Boot, per una maggiore personalizzazione e estensibilità dell'applicazione.

Mentre uno starter può semplificare la configurazione iniziale, una libreria come SpringLine 2 offre una maggiore flessibilità, controllo e ottimizzazione, rendendola una scelta ideale per progetti più complessi e personalizzati.

2. Obiettivo

SpringLine 2 mira a fornire una piattaforma robusta e flessibile. Mentre SpringBoot stabilisce un solido fondamento, SpringLine 2 va oltre, permettendo una personalizzazione e un'ottimizzazione granulare. Questo si traduce in una maggiore integrazione, scalabilità, sicurezza e mantenibilità.

Con la sua architettura modulare, SpringLine 2 facilita l'interoperabilità tra diverse piattaforme e tecnologie. Questo garantisce che le applicazioni non siano solo funzionali, ma che siano anche progettate per resistere ai cambiamenti futuri, riducendo la necessità di aggiornamenti costosi. In altre parole, le applicazioni sviluppate con SpringLine 2 sono "future-proof", pronte ad adattarsi nel tempo nonostante la fisiologica evoluzione tecnologica.

I punti di forza di SpringLine 2 includono:

Personalizzazione: a differenza delle configurazioni di base che offrono impostazioni predefinite e generali, le estensioni di SpringLine 2 permettono di raffinare e modificare l'applicazione in base alle esigenze;

Sicurezza: la personalizzazione avanzata delle configurazioni consente di implementare misure di sicurezza robuste rispetto agli standard Aria S.p.A;

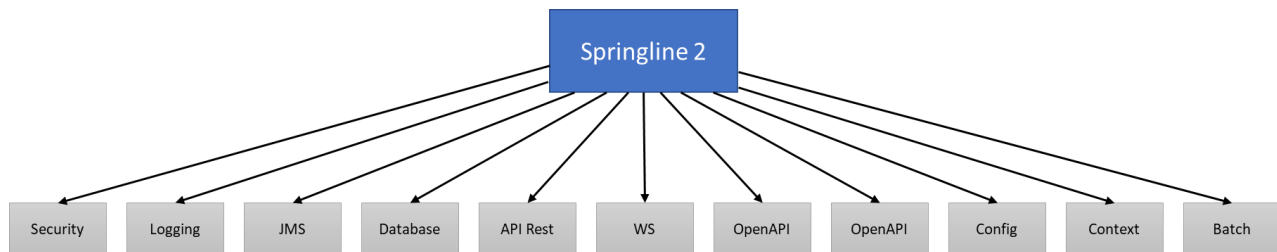
Interoperabilità: Le estensioni garantiscono che l'applicazione funzioni in modo ottimale nei vari ambienti e con diverse tecnologie;

Conformità agli standard: SpringLine 2 assicura che le applicazioni siano conformi agli standard qualitativi e funzionali stabiliti da Aria S.p.A.

Estensione delle capacità di SpringBoot: oltre alle potenzialità di base di SpringBoot, la libreria offre strumenti e funzionalità aggiuntive per una maggiore personalizzazione e ottimizzazione;

In sintesi, SpringLine 2 amplifica le potenzialità intrinseche di SpringBoot, fornendo agli sviluppatori una serie di strumenti e funzionalità aggiuntive che permettono di costruire applicazioni più robuste, efficienti e conformi agli elevati standard di qualità e sicurezza definiti da Aria S.p.A.

3. Diagramma delle componenti

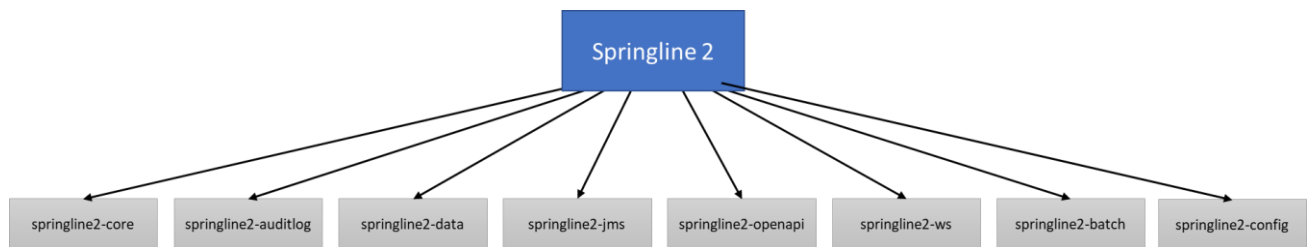


Dal punto di vista funzionale, SpringLine 2 si compone dei seguenti moduli:

- **Security:** Si occupa dell'integrazione con diversi Identity Provider, garantendo la conformità agli elevati standard di sicurezza.
- **Logging:** Fornisce capacità di registrazione conformi agli standard di Aria S.p.A.;
- **JMS:** Modulo specifico per il Java Message Service.
- **Database:** Gestisce la connettività e l'interazione con i database.
- **API Rest:** Modulo dedicato alla creazione e gestione delle API RESTful.
- **WS:** Modulo per la gestione dei Web Services.
- **OpenAPI:** Fornisce strumenti per la documentazione e definizione delle API.
- **Batch:** Fornisce strumenti per l'utilizzo di spring batch.
- **Config:** Si occupa della gestione della configurazione integrandosi con Hashicorp Consul e Cyberak Conjur.
- **Context:** Modulo centrale per la gestione del contesto applicativo.

4. Dipendenze Applicative

Nella progettazione, lo sviluppatore dovrà considerare la seguente struttura modulare aggiungendo nel pom applicativo le rispettive dipendenze. La scomposizione modulare differisce da quella funzionale:



```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-core</artifactId>
  <version>${spring-line.version}</version>
</dependency>
```

```
<!-- SPRINGLINE2 DATA: per uso di JPA -->
```

```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-data</artifactId>
  <version>${spring-line.version}</version>
</dependency>
```

```
<!-- SPRINGLINE2 JMS: per uso di JMS -->
```

```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-jms</artifactId>
  <version>${spring-line.version}</version>
</dependency>
```

```
<!-- SPRINGLINE2 OPENAPI: per uso di OpenAPI -->
```

```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-openapi</artifactId>
  <version>${spring-line.version}</version>
</dependency>
```

```
<!-- SPRINGLINE2 WS: per uso di client WS -->
```

```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-ws</artifactId>
  <version>2026.01.01</version>
</dependency>
```

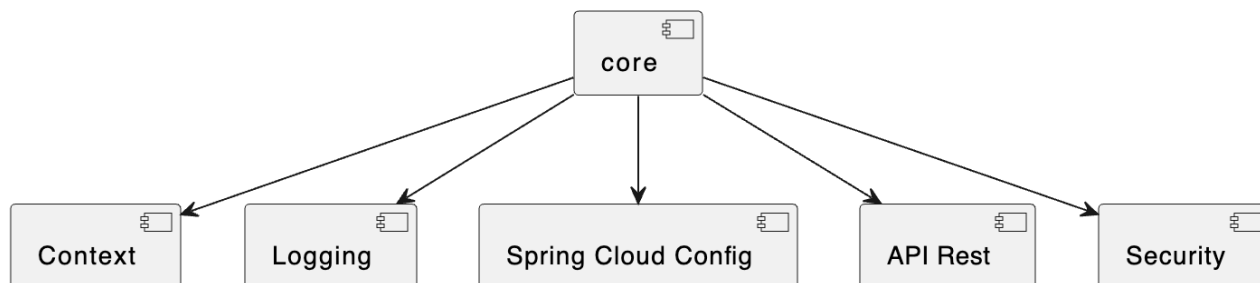
```
<!-- SPRINGLINE2 BATCH: per uso di SpringBatch dalle versioni 2024.02.01 e 2023.03.01 -->
```

```
<dependency>
  <groupId>it.ariaspa.springline2</groupId>
  <artifactId>springline2-batch</artifactId>
  <version>${spring-line.version}</version>
</dependency>
```

```
<!-- SPRINGLINE2 CONFIG: per l'uso di Cconsul e Conjur. Dalle versioni 2025.02.01 e  
2024.02.01 -->  
<dependency>  
  <groupId>it.ariaspa.springline2</groupId>  
  <artifactId>springline2-config</artifactId>  
  <version>${spring-line.version}</version>  
</dependency>
```

* Per le versioni vedere l'appendice A

5. SpringLine 2 – Core

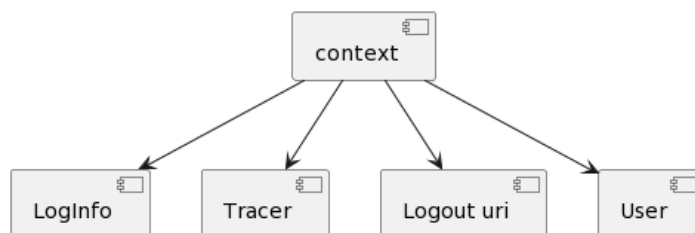


Nell'architettura di SpringLine 2, il "Core" si posiziona come il modulo principale su cui poggia l'intera soluzione. Esso fornisce le strutture fondamentali e le componenti basilari necessarie per sostenere e integrare gli altri moduli. Agendo come pilastro centrale di SpringLine 2, il modulo "Core" assicura un'architettura solida, coesa e resiliente. Ogni altro modulo, sia esso dedicato alla sicurezza, alla gestione dei dati o alle comunicazioni, si innesta su questo nucleo centrale, beneficiando della sua robustezza e flessibilità. La sua integrazione è essenziale in ogni progetto applicativo, garantendo che l'architettura mantenga coerenza, scalabilità e affidabilità.

Le principali funzionalità offerte dal modulo "Core" includono:

- **Context:** una delle caratteristiche distintive del modulo "Core" è la sua capacità di gestire e mantenere un contesto per ogni transazione;
- **Logging:** fornisce strumenti di logging, permettendo agli sviluppatori di monitorare, registrare e analizzare le attività dell'applicazione. Questo facilita la diagnosi e la risoluzione di problemi, garantendo al contempo una tracciabilità completa delle operazioni;
- **Spring Cloud Config:** gestisce la lettura delle configurazioni applicative in modo centralizzato dal configuration server. A differenza di quanto offerto da Spring Cloud Config, SpringLine 2 aggiunge la gestione di una cache locale;
- **API Rest:** guida il programmatore nell'esporre e nel chiamare API Rest, in linea con gli standard definiti nelle guide Aria S.p.A.;
- **Security:** la sicurezza è una priorità nell'architettura di SpringLine 2. Il modulo "Core" integra meccanismi di sicurezza robusti e conformi agli standard Aria S.p.A., assicurando che le applicazioni siano protette.

5.1 SpringLine 2 – Context



Il context in SpringLine 2 rappresenta un pilastro fondamentale. Esso è stato progettato con l'obiettivo di gestire, conservare e rendere facilmente accessibili tutte le informazioni relative ad ogni transazione effettuata.

Ogni qualvolta si avvia una transazione, SpringLine 2 crea automaticamente un contesto dedicato. Tuttavia, è fondamentale sottolineare un'eccezione a questo meccanismo: in artefatti di tipo batch che dipendono da SpringLine 2, il contesto non viene inizializzato in modo automatico. In tali scenari, spetta agli sviluppatori inizializzarlo manualmente al fine di assicurare una corretta esecuzione dell'applicazione evitando potenziali incongruenze o errori.

Di seguito un esempio Java che mostra come creare un contesto (valido negli scenari in cui SpringLine 2 non ne genera uno):

```

@GetMapping("/method")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.NO_CONTENT)
@Operation(description = "Metodo di test", responses = {
    @ApiResponse(
        description = "Successful Operation",
        responseCode = "204") })
public ResponseEntity<Void> getMethod() {
    ContextHolder.attach();
    Context context = ContextHolder.currentContext();
    return RestResponse.noContent();
}
  
```

L'accesso alle informazioni contenute nel contesto può avvenire attraverso due modalità principali:

- **Accesso Manuale:** questa modalità è sicuramente la più flessibile in quanto offre agli sviluppatori la libertà di intervenire direttamente e recuperare le informazioni necessarie in qualsiasi punto dell'applicativo;
- **Annotation:** SpringLine 2 fornisce un Parameter Annotation che permette di accedere in modo semplificato e strutturato alle informazioni contenute nel contesto. SpringLine 2 si occupa di iniettare automaticamente il valore corrispondente ma sarà responsabilità dello

sviluppatore mascherare il parametro per evitare che venga interpretato come un input diretto del servizio esposto.

Il contesto, acceduto in una delle due modalità definite precedentemente, consente agli sviluppatori di recuperare:

- **Utente Loggato:** questa funzione consente di accedere alle informazioni dell'utente che ha effettuato l'autenticazione.
- **LogInfo:** si tratta di un insieme di dettagli specifici per il logging;
- **Tracer:** questo strumento è stato progettato per tracciare in modo dettagliato le operazioni di ogni transazione. Per ognuna di essa, il Tracer viene configurato come segue:
 - **traceld:** si tratta dell'identificativo della transazione principale che rimane invariato per tutta la sua durata.
 - **spanId:** Questo identificativo, specifico per le transazioni secondarie, è inizializzato con il valore "0000000000000000".

Quando si avvia una nuova transazione secondaria, il traceld rimane inalterato, mentre lo spanId riceve un nuovo identificativo unico. Grazie al traceld, è possibile tracciare l'intero percorso di una transazione, comprese tutte le sue transazioni secondarie correlate.

- **Log-Out Url:** si tratta di un URL dedicato specificamente alla gestione del logout utente. È disponibile solo in alcuni sistemi di autenticazione. Per questi, lo sviluppatore può impostare tale uri al fine di reindirizzare la richiesta solo dopo che l'utente ha completato la sua operatività sulla piattaforma in uso. I sistemi di autenticazione che contemplano il log-out uri sono: **SAMWEB – IDPC**

Dalle versioni 2026.01.01 e 2025.04.01 è stata introdotto la possibilità di associare e gestire dati applicativi arbitrari all'interno del contesto corrente (Context), offrendo maggiore flessibilità nello scambio di informazioni tra le componenti dell'applicazione.

Modifiche Principali:

- Estensione della classe Context:
La classe Context è stata modificata per includere una struttura dati che permette di memorizzare informazioni aggiuntive non predefinite.
- Supporto per Dati Applicativi tramite Mappa:
È stata aggiunta una mappa interna (`private final Map<String, Object> attributes`) alla classe Context. Questa mappa serve a contenere dati applicativi generici, identificati da una chiave di tipo String.
- Accesso alla Mappa degli Attributi:
È stato introdotto un nuovo metodo pubblico:

- `public Map<String, Object> getAttributes()`: Questo metodo restituisce la mappa interna che può essere utilizzata per inserire, recuperare o rimuovere dati applicativi associati al contesto corrente.

Questa modifica estende significativamente le capacità della classe `Context`, permettendo agli sviluppatori di associare e gestire facilmente informazioni specifiche del business o della request all'interno dello stesso contesto di esecuzione (thread/request). Le funzionalità esistenti relative alla gestione dell'utente, del log, del tracer e del tempo DB rimangono invariate.

Per aggiungere o recuperare dati applicativi dal contesto, ottieni la mappa degli attributi tramite il nuovo metodo `getAttributes()` oppure l'annotation `@CtxAttributes` e utilizza i metodi standard della classe `java.util.Map` (come `put()`, `get()`, `remove()`) su di essa.

Tramite il metodo `getAttributes()`:

Recupera l'oggetto `Context` e chiama `getAttributes()`:

```
Context context = ContextHolder.currentContext();
Map<String, Object> appData = context.getAttributes();

// Aggiungere un dato
appData.put("idOrdine", 12345);
appData.put("nomeUtenteLoggato", "Mario Rossi");

// Recuperare un dato
Integer idOrdine = (Integer) appData.get("idOrdine");
String nomeUtente = (String) appData.get("nomeUtenteLoggato");
```

Tramite l'annotation `@CtxAttributes`:

Applica l'annotation a un parametro di tipo `Map<String, Object>` in un tuo metodo (es. un metodo di un controller o di un servizio):

```
public void processRequest(@CtxAttributes Map<String, Object> requestAttributes) {
    requestAttributes.put("elaborato", true);

    Boolean processed = (Boolean) requestAttributes.get("elaborato");
}
```

Note:

Questa è una modifica additiva. L'uso del metodo `getAttributes()` e l'interazione con la mappa restituita sono opzionali e non impatta il codice esistente che utilizza solo le funzionalità precedenti della classe `Context`.

5.1.1 Accesso al Context

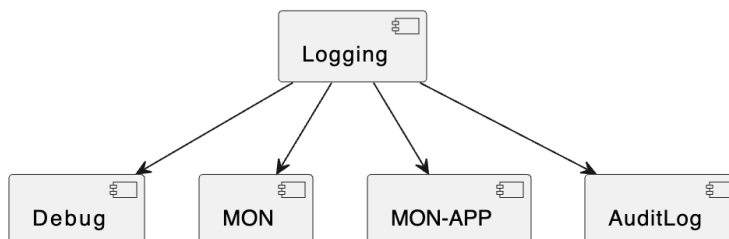
Gli esempi di codice Java riportati di seguito, mostrano allo sviluppatore come recuperare il contesto in SpringLine 2 mediante:

- un accesso manuale, utilizzabile in qualsiasi fase del flusso applicativo
- un accesso tramite Parameter Annotation (**@CtxContext**)

```
@GetMapping("/method")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.NO_CONTENT)
@Operation(description = "Metodo di test", responses = {
    @ApiResponse(
        description = "Successful Operation",
        responseCode = "204") })
public ResponseEntity<Void> getMethod() {
    Context context = ContextHolder.currentContext();
    ...
    ...
    return RestResponse.noContent();
}

@GetMapping("/method")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.NO_CONTENT)
@Operation(description = "Metodo di test", responses = {
    @ApiResponse(
        description = "Successful Operation",
        responseCode = "204") })
public ResponseEntity<Void> getMethod(@Parameter(hidden = true)
@CtxContext Context context) {
    ...
    ...
    return RestResponse.noContent();
}
```

5.2 SpringLine 2 – Logging



Il sistema di logging svolge un ruolo cruciale per assicurare tracciabilità, diagnosi e monitoraggio delle operazioni eseguite all'interno delle applicazioni. Esso è stato progettato per fornire una visione dettagliata e strutturata delle attività dell'applicazione attraverso diversi livelli di log:

1. **Debug**: Questo log rappresenta l'ambiente più flessibile, prevalentemente utilizzato dagli sviluppatori durante le fasi di sviluppo e debug. SpringLine 2 non interviene direttamente in questo log, garantendo agli sviluppatori completa libertà su ciò che viene registrato.
2. **MON (Monitoraggio)**: Questo log è progettato per tracciare tutte le interazioni in ingresso e in uscita dell'applicazione su protocollo HTTP. Ogni interazione, come l'invocazione di un metodo di un controller, viene registrata nel log MON.
3. **MON-APP (Monitoraggio Applicativo)**: Questo log è specificamente progettato per tracciare eventi e operazioni a livello applicativo. Esso fornisce una visione dettagliata delle attività specifiche dell'applicazione, garantendo una tracciabilità accurata delle operazioni.
4. **AuditLog**: Questo log è dedicato alla registrazione di eventi di particolare rilevanza all'interno dell'applicazione. Funziona come un registro formale delle attività e deve essere usato per scopi di conformità e sicurezza. Per approfondimenti consultare il manuale d'uso rilasciato da Aria S.p.A.

In sintesi, il sistema di logging di SpringLine 2 Core è stato progettato per offrire agli sviluppatori una suite completa di strumenti per monitorare, diagnosticare e tracciare le attività all'interno delle loro applicazioni in modo efficace e strutturato.

Dalle versioni 2024.05.01 e 2023.06.01 di SpringLine 2:

- se il log della classe `it.ariaspa.springline2.filter.TracerFilter` viene impostato a `DEBUG` vengono loggate informazioni della request.
- è stata aggiunta la variabile (traceId) MDC e valorizzata con i valori di trace
- è stato modificato il pattern layout del log in console e file per stampare la variabile MDC traceId.

Il sistema di logging di SpringLine 2 è basato su logback. Sono presenti i seguenti file di configurazione:

- **logback-springline2.xml**

È la configurazione di default di Springline 2 che prevede che vengano generati in file separati i log MON, APP, AUDIT e DEBUG. Quest'ultimo viene scritto nello stesso file insieme ai log di console. Nell'appendice E è presente la configurazione completa.

- **logback-springline2-ocp.xml**

È la configurazione specifica per le applicazioni containerizzate in Openshift di ARIA. Questa configurazione prevede di scrivere tutti i log in console e ogni riga di log è preceduta da un prefisso per identificarne il tipo:

- MON: MON->
- APP: APP->
- AUDIT: AUDIT->
- DEBUG: *senza prefisso*

Nell'appendice E è presente la configurazione completa.

- **logback-springline2-encrypt.xml** (dalle versioni 2025.02.01 e 2024.07.01)

È come la configurazione **logback-springline2.xml** con l'aggiunta della cifratura dei log MON, APP, AUDIT (vedere capitolo "Cifratura dei log"). Nell'appendice E è presente la configurazione completa.

- **logback-springline2-ocp-encrypt.xml** (dalle versioni 2025.02.01 e 2024.07.01)

È come la configurazione **logback-springline2-ocp.xml** con l'aggiunta della cifratura dei log MON, APP, AUDIT (vedere capitolo "Cifratura dei log"). Nell'appendice E è presente la configurazione completa.

5.2.1 MON (Log di monitoraggio)

Il MON monitora tutte le interazioni in entrata e in uscita dell'applicazione. Per garantire il tracciamento, è essenziale che gli sviluppatori utilizzino gli strumenti forniti da SpringLine 2: RestTemplate, WebServiceTemplate e SplJmsTemplate. Durante ogni interazione, nel MON vengono creati due log: uno per l'ingresso e uno per l'uscita, contenenti dettagli tecnici dell'operazione.

Di seguito vengono riportate le informazioni che vengono tracciate nel log:

5.2.1.1 HTTP - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta HTTP, popolato con System.currentTimeMillis()
server	Host Name

Proprietà	Valore
app	Application Name
met	Method + “#” + Request uri
ver	“N/A”
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>]” altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	“REST”
rv	Circuit dell'oggetto User

5.2.1.2 HTTP - OUT

Proprietà	Valore
timestamp	Tempo di fine della richiesta HTTP, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Method + “#” + Request uri
ver	“N/A”
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>]” altrimenti. Appld è contenuta nell'oggetto Tracer del Context

Proprietà	Valore
canale	"REST"
rv	Circuit dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
tpDb	TimeDb dell'oggetto Context (esprime il tempo di esecuzione all'interno della stessa transazione per l'accesso alla fonte dati)
extra	Informazione dell'oggetto MonExtra
esito	StackTrace dell'oggetto Throwable se si è verificata una eccezione. Status della response altrimenti
esito1	E' valorizzato solo se si è verificata una eccezione con il Message dell'oggetto Throwable

5.2.1.3 Client Rest - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta REST, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Method + "#" + Request uri
ver	"N/A"
idDc	TraceId + "," + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	"CREATO" se il campo ParentSpanId = "0000000000000000" "PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	"REST"
rv	Circuit dell'oggetto User
client	Name della Metaclass della classe RestTemplate

5.2.1.4 Client Rest - OUT

Proprietà	Valore
-----------	--------

Proprietà	Valore
timestamp	Tempo di fine della richiesta REST, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Method + “#” + Request uri
ver	“N/A”
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	“REST”
rv	Circuit dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
client	Name della Metaclassa della classe RestTemplate
esito	StackTrace dell'oggetto Throwable se si è verificata una eccezione. StatusCode della response altrimenti

5.2.1.5 JMS LISTENER - IN

Proprietà	Valore
timestamp	Tempo di inizio dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Destinazione del messaggio JMS
ver	“N/A”
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User

Proprietà	Valore
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	"CREATO" se il campo ParentSpanId = "0000000000000000" "PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	"JMS"
rv	Circuit dell'oggetto User

5.2.1.6 JMS LISTENER - OUT

Proprietà	Valore
timestamp	Tempo di fine dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Destinazione del messaggio JMS
ver	"N/A"
idDc	TraceId + " ," + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	"CREATO" se il campo ParentSpanId = "0000000000000000" "PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	"JMS"
rv	Circuit dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
tpDb	TimeDb dell'oggetto Context (esprime il tempo di esecuzione all'interno della stessa transazione per l'accesso alla fonte dati)
extra	Informazione dell'oggetto MonExtra
esito	StackTrace dell'oggetto Throwable se si è verificata una eccezione. "OK" della response altrimenti

Proprietà	Valore
esito1	E' valorizzato solo se si è verificata una eccezione con il Message dell'oggetto Throwable
liv	"INFO"

5.2.1.7 JMS Client - IN

Proprietà	Valore
timestamp	Tempo di inizio dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Destinazione del messaggio JMS
ver	"N/A"
idDc	TraceId + "," + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	"CREATO" se il campo ParentSpanId = "0000000000000000" "PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	"JMS"
rv	Circuit dell'oggetto User
client	Name della Metaclassa della classe SplJmsTemplate

5.2.1.8 JMS Client - OUT

Proprietà	Valore
timestamp	Tempo di fine dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	Destinazione del messaggio JMS
ver	"N/A"

Proprietà	Valore
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	“JMS”
rv	Circuit dell'oggetto User
client	Name della Metaclass della classe SplJmsTemplate
esito	“OK”
liv	“INFO”

5.2.1.9 WS - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta SOAP, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	LocalName del FirstChild del SOAPBody
ver	Namespace Uri del FirstChild del SOAPBody
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	“WS”

Proprietà	Valore
rv	Circuit dell'oggetto User
client	Name della Metaclassa della classe WebServiceTemplate

5.2.1.10 WS - OUT

Proprietà	Valore
timestamp	Tempo di fine della richiesta SOAP, popolato con System.currentTimeMillis()
server	Host Name
app	Application Name
met	LocalName del FirstChild del SOAPBody
ver	Namespace Uri del FirstChild del SOAPBody
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
user	Username dell'oggetto User
ruolo	Role dell'oggetto User
str	Organization dell'oggetto User
infolDc	“CREATO” se il campo ParentSpanId = “0000000000000000” “PROPAGATO[<appld>] altrimenti. Appld è contenuta nell'oggetto Tracer del Context
canale	“REST”
rv	Circuit dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
client	Name della Metaclassa della classe WebServiceTemplate
esito	StackTrace dell'oggetto Exception se si è verificata una eccezione. “OK” della response altrimenti

Gli sviluppatori hanno la libertà di personalizzare ulteriormente il MON. Possono, infatti, integrare informazioni aggiuntive attraverso **MonExtra**. Le principali sono: esito1, esito2 e infoApp. Per farlo, gli sviluppatori devono accedere al Context. L'accesso è descritto nel paragrafo 5.1.1

Ecco un esempio pratico in Java:

```
@GetMapping("/method")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.NO_CONTENT)
```



```

@Operation(description = "Metodo di test", responses = {
    @ApiResponse(
        description = "Successful Operation",
        responseCode = "204") })
public ResponseEntity<Void> getMethod() {
    final LogInfo logInfo =
        ContextHolder.currentContext().getLogInfo();
    logInfo.getMonExtra().setEsito1("Esito 1");
    logInfo.getMonExtra().setEsito2("Esito 2");
    logInfo.getMonExtra().setInfoApp("InfoApp");

    return RestResponse.noContent();
}

```

Di seguito sono elencate le properties (specificate nei file yaml/properties) che possono essere personalizzate in base al canale di attivazione del log MON.

5.2.1.11 HTTP

Proprietà	Valore
spl.http.logging.mon.enabled	True/False Valore di default True
spl.http.logging.mon.uri-matchers	Valore di default: /** Dominio di valori: String Identifica gli uri per i quali verrà effettuato il log. E' possibile definire l'elenco degli end point separati da ","
spl.http.logging.mon.excluded-uri-matchers	Valore di Default: /favicon.ico Dominio di valori: String Identifica gli uri per i quali il log non sarà tracciato. E' possibile definire l'elenco degli end point separati da ","

5.2.1.12 JMS - LISTENER

Proprietà	Valore
spl.jms.logging.mon.enabled	True/False Valore di default True
spl.jms.logging.mon.destination-matchers	Valore di default: ** Dominio di valori: String Identifica le code per le quali sarà tracciato il log E' possibile definire l'elenco degli end point separati da ","
spl.jms.logging.mon.excluded-destination-matchers	Valore di Default: Stringa vuota Dominio di valori: String Identifica le code per i quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da ","

5.2.1.13 Client Rest

Proprietà	Valore
-----------	--------

Proprietà	Valore
spl.client-rest.logging.mon.enabled	True/False Valore di default True
spl.client-rest.logging.mon.uri-matchers	Valore di default: /** Dominio di valori: String Identifica gli uri per i quali verrà effettuato il log. E' possibile definire l'elenco degli end point separati da “,”
spl.client-rest.logging.mon.excluded-uri-matchers	Valore di Default: Stringa Vuota Dominio di valori: String Identifica gli end point per i quali il log non sarà tracciato. E' possibile definire l'elenco degli end point separati da “,”

5.2.1.14 JMS Client

Proprietà	Valore
spl.client-jms.logging.mon.enabled	True/False Valore di default True
spl.client-jms.logging.mon.destination-matchers	Valore di Default ** Dominio di valori: String Identifica le code per le quali sarà tracciato il log E' possibile definire l'elenco delle code separate da “,”
spl.client-jms.logging.mon.excluded-destination-matchers	Valore di Default: Stringa vuota Dominio di valori: String Identifica le code per i quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da “,”

5.2.1.15 WS Client

Proprietà	Valore
spl.client-ws.logging.mon.enabled	True/False Valore di default True
spl.client-ws.logging.mon.method-matchers	Valore di Default ** Dominio di valori: String Identifica tutti i remote method per le quali sarà tracciato il log. E' possibile definire l'elenco delle code separate da “,”
spl.client-ws.logging.mon.excluded-method-matchers	Valore di Default: Stringa Vuota Dominio di valori: String Identifica i remote method per i quali il log non sarà tracciato. E' possibile definire l'elenco dei method separati da “,”

5.2.2 MON-APP (Log di monitoraggio applicativo)

Il MON-APP, al pari del MON, traccia tutte le interazioni dell'applicazione in uscita. Tuttavia, la peculiarità del MON-APP è quella di arricchire il log del MON registrando il body delle response. Ogni interazione lega il MON ed il MON-APP mediante una property: idDc, che funge da identificativo univoco di transazione. Tale property è valorizzata come concatenazione del Traceld e SpanId dell'oggetto Tracer contenuto nel Context. Questo idDc viene generato alla prima richiesta e successivamente propagato a tutte le transazioni secondarie. Questo meccanismo

permette di analizzare i log sia del MON che del MON-APP per ottenere una visione completa del flusso applicativo, dalle informazioni generali ai dettagli più specifici.

Di seguito vengono riportate le informazioni che vengono tracciate nel log:

5.2.2.1 HTTP - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta HTTP, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
in	Stream di input della richiesta HTTP

5.2.2.2 HTTP - OUT

Proprietà	Valore
timestamp	Tempo di fine della richiesta HTTP, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
out	Stream di output della risposta HTTP
extra	Informazioni dell'oggetto AppExtra

5.2.2.3 Client Rest - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta REST, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
client	Name della Metaclass della classe RestTemplate

Proprietà	Valore
in	Body della richiesta come array di byte

5.2.2.4 Client Rest - OUT

Proprietà	Valore
timestamp	Tempo di fine della richiesta REST, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
client	Name della Metaclassa della classe RestTemplate
out	Body Response se presente

5.2.2.5 JMS LISTENER - IN

Proprietà	Valore
timestamp	Tempo di inizio dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
in	Bytes relativo al contenuto del messaggio JMS Bytes dell'eccezione altrimenti

5.2.2.6 JMS LISTENER - OUT

Proprietà	Valore
timestamp	Tempo di fine dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione

Proprietà	Valore
out	Bytes relativo al contenuto del messaggio JMS Bytes dell'eccezione altrimenti
extra	Informazioni dell'oggetto AppExtra

5.2.2.7 JMS Client - IN

Proprietà	Valore
timestamp	Tempo di inizio dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
client	Name della Metaclassa della classe SplJmsTemplate
in	Bytes relativo al contenuto del messaggio JMS

5.2.2.8 JMS Client - OUT

Proprietà	Valore
timestamp	Tempo di fine dell'operazione JMS, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
client	Name della Metaclassa della classe SplJmsTemplate
out	Bytes relativo al contenuto del messaggio JMS

5.2.2.9 WS - IN

Proprietà	Valore
timestamp	Tempo di inizio della richiesta SOAP, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + “,” + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User

Proprietà	Valore
client	Name della Metaclassa della classe WebServiceTemplate
in	Array di Byte relativo al corpo della richiesta SOAP

5.2.2.10 WS - OUT

Proprietà	Valore
timestamp	Tempo di fine della richiesta SOAP, popolato con System.currentTimeMillis()
server	Host Name
idDc	TraceId + "," + SpanId (entrambe le informazioni appartengono all'oggetto Tracer del Context)
idCoop	TrackId dell'oggetto User
elapsed	Differenza temporale (espressa in millisecondi) tra l'inizio e la fine della transazione
client	Name della Metaclassa della classe WebServiceTemplate
out	Array di Byte relativo al corpo della risposta SOAP

Inoltre, come per il MON, gli sviluppatori hanno la possibilità di personalizzare ulteriormente il log del MON-APP. Utilizzando il Context, possono accedere ad AppExtra e definire la property infoApp per aggiungere informazioni supplementari.

```

@GetMapping("/method")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.NO_CONTENT)
@Operation(description = "Metodo di test", responses = {
    @ApiResponse(
        description = "Successful Operation",
        responseCode = "204") })
public ResponseEntity<Void> getMethod() {
    final LogInfo logInfo =
        ContextHolder.currentContext().getLogInfo();
    logInfo.getAppExtra().setInfoApp("Info App");

    return RestResponse.noContent();
}

```

Di seguito sono elencate le properties (specificate nei file yaml/properties) che possono essere personalizzate in base al canale di attivazione del log MON-APP.

5.2.2.11 HTTP

Proprietà	
spl.http.logging.app.enabled	True/False Valore di default False
spl.http.logging.app.uri-matchers	Valore di Default: /** Dominio di valori: String Identifica gli uri per i quali sarà tracciato il log E' possibile definire l'elenco degli uri separati da “,”
spl.http.logging.app.excluded-uri-matchers	Valore di Default: /favicon.ico Dominio di valori: String Identifica gli uri per i quali il log non sarà tracciato. E' possibile definire l'elenco degli uri separati da “,”
spl.http.logging.app.excluded-content-types	Valore di Default: Empty Dominio di valori: String Identifica l'elenco dei Content-types per i quali il log non sarà tracciato. Ogni elemento è separato da “,”
spl.http.logging.app.body-max-length	Valore di Default: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato
spl.http.logging.app.add-request-uri ¹	Valore di Default: false Dominio di valori: true/false Abilita la scrittura nel campo IN nel formato <i>httpMethod#requestUri#queryString#body</i>

5.2.2.12 JMS - LISTENER

Proprietà	
spl.jms.logging.app.enabled	True/False Valore di default False
spl.jms.logging.app.destination-matchers	Valore di Default: ** Dominio di valori: String Identifica le code per i quali sarà tracciato il log E' possibile definire l'elenco delle code separate da “,”
spl.jms.logging.app.excluded-destination-matchers	Valore di Default: String Empty Dominio di valori: String Identifica le code per le quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da “,”
spl.jms.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

5.2.2.13 Client Rest

Proprietà	
spl.client-rest.logging.app.enabled	True/False Valore di default False

¹ Dalle versioni 2025.05.1 e 2026.02.01

Proprietà	
spl.client-rest.logging.app.uri-matchers	Valore di Default: /** Dominio di valori: String Identifica gli end point per i quali sarà tracciato il log E' possibile definire l'elenco degli end point separate da ","
spl.client-rest.logging.app.excluded-uri-matchers	Valore di Default: String Empty Dominio di valori: String Identifica gli end point per i quali il log non sarà tracciato. E' possibile definire l'elenco degli uri separati da ","
spl.client-rest.logging.app.excluded-content-types	Valore di Default: Empty Dominio di valori: String Identifica l'elenco dei Content-types per i quali il log non sarà tracciato. Ogni elemento è separato da ","
spl.client-rest.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

5.2.2.14 JMS Client

Proprietà	
spl.client-jms.logging.app.enabled	True/False Valore di default False
spl.client-jms.logging.app.destination-matchers	Valore di Default: ** Dominio di valori: String Identifica le code per i quali sarà tracciato il log E' possibile definire l'elenco delle code separate da ","
spl.client-jms.logging.app.excluded-destination-matchers	Valore di Default: String Empty Dominio di valori: String Identifica le code per le quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da ","
spl.client-jms.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

5.2.2.15 WS Client

Proprietà	
spl.client-ws.logging.app.enabled	True/False Valore di default False
spl.client-ws.logging.app.method-matchers	Valore di Default: ** Dominio di valori: String Identifica i remote method per i quali sarà tracciato il log. E' possibile definire l'elenco dei remote method separati da ","

Proprietà	
spl.client-ws.logging.app.excluded-method-matchers	Valore di Default: String Empty Dominio di valori: String Identifica i remote method per i quali il log non sarà tracciato. E' possibile definire l'elenco dei remote method separate da “,”
spl.client-ws.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per il quale il log sarà tracciato

5.2.3 AuditLog

I log di business sono essenziali per l'auditing. Aria S.p.A ha stabilito le linee guida per la registrazione dei log di Audit. Per una comprensione approfondita, si suggerisce di fare riferimento alla linea guida specifica. Sebbene SpringLine 2 gestisca autonomamente la registrazione di determinati eventi, ve ne sono alcuni legati strettamente all'applicazione. In questi casi, è richiesta l'attenzione e l'azione diretta dello sviluppatore. L'accesso ai log di Audit avviene attraverso il contesto. Ottenuti gli oggetti, sarà responsabilità dello sviluppatore completare il popolamento delle informazioni rispetto all'evento specifico da monitorare. Di seguito, un esempio in Java che illustra come recuperare i log di audit dal contesto:

```
final LogInfo logInfo =
    ContextHolder.currentContext().getLogInfo();
final AuditEvent logEvent = logInfo.getAuditEvent();
final AuditNetwork logNetwork = logInfo.getAuditNetwork();
final AuditUser logUser = logInfo.getAuditUser();
final AuditTransaction logTransaction =
    logInfo.getAuditTransaction();

AuditLog auditLogInstance = AuditLog.getInstance();

// valorizzazione delle property
// valorizzazione delle property
// valorizzazione delle property
// valorizzazione delle property

auditLogInstance.log(logTransaction, logEvent, logNetwork,
    logUser);
```

Di seguito, sono riportate le property pre-valorizzate da SpringLine 2 a seguito di chiamate http:

Nodo	Valore
AuditEvent	

Nodo	Valore
RecourseUrl	request.getRequestURI()
AccessMode	"HTTP"
AuditTransaction	
CodApp	spring.application.name
Method	request.getMethod() + "#" + request.getRequestURI()
IdInteraction	tracer.getTraceld() + "," + tracer.getSpanId());
SystemName	Nome Host
Rv	Se utente autenticato "R" o "V" altrimenti "R"
AuditUser	
SrcUser	Se utente autenticato Username altrimenti "ANONYMOUS"
DstUser	Se utente autenticato Username altrimenti "ANONYMOUS"
Username	Se utente autenticato FamilyName+", "+Name altrimenti "N/A"
UserRole	Se utente autentica Role
UserType	Se utente non autenticato "ANONYMOUS"
UserStr	Se utente autenticato Organization
AuthChannel	Se utente autenticato Channel altrimenti "NO_CHANNEL"
AuthMethod	Se utente autenticato Method altrimenti "NO_METHOD"
AuditNetwork	
SrcIp	request.getRemoteAddr()
RequestURL	request.getRequestURL()
UserAgent	request.getUserAgent()

SpringLine 2 provvede a tracciare in automatico una serie di eventi che sono di seguito indicati:

5.2.3.1 Start/ShutDown Application

Nodo	Valore
AuditEvent	
Id	"app_startup" / "app_shutdown"
EventDesc	spring.application.name + " startup" oppure spring.application.name + " shutdown"
EventLevel	1

Nodo	Valore
EventType	0
PrivResource	"N"
Status	"success"
AccessMode	"DIRECTCALL"
AuditTransaction	
Priority	108
CodApp	spring.application.name
Method	"N/A"
IdInteraction	"N/A"
SystemName	Nome Host
AuditUser	
SrcUser	"ANONYMOUS"
DstUser	"ANONYMOUS"
Username	"N/A"
UserRole	"R"
UserType	"ANONYMOUS"
PrivUser	"N"
AuthChannel	"NO_CHANNEL"
AuthMethod	"NO_METHOD"
AuditNetwork	
SrcIp	Host Address

5.2.3.2 Utente non Autorizzato

Nodo	Valore
AuditEvent	
Id	"authz_fail"
EventDesc	"Utente non autorizzato"
EventLevel	3
EventType	1
PrivResource	"N"
Status	"failure"
Reason	"Forbidden"

Nodo	Valore
AuditUser	
PrivUser	"N"

5.2.3.3 JWT – Utilizzo Token di autorizzazione

Nodo	Valore
AuditEvent	
Id	"authn_token_use"
EventDesc	"Utilizzo del token di autorizzazione"
EventLevel	3
EventType	0 (NORMAL) - In caso di eccezione 1 (ANOMALY)
PrivResource	"N"
Status	"success" - In caso di eccezione "failure"
EventData	"token" + Id
Reason	"Forbidden" – In caso di eccezione Exception Message
AuditUser	
PrivUser	"N"
AuditTransaction	Nodo pre-valorizzato solo in caso di eccezione
Priority	106

5.2.3.4 Eccezione inaspettata

Nodo	Valore
AuditEvent	
Id	"unexpected_exception"
EventDesc	Exception Message
EventLevel	2
EventType	2
PrivResource	"N"
Status	"failure"
AuditUser	
PrivUser	"N"
AuditTransaction	

Nodo	Valore
Priority	108

5.2.3.5 Eccezione validazione dati

Per le tre eccezioni riportate di seguito, i log di Audit sono configurati in modo analogo, ad eccezione della proprietà EventData la cui valorizzazione sarà puntualmente descritta. Nella tabella invece, vengono elencate le proprietà comuni.

MethodArgumentTypeMismatchException: Property name + Exception Message

ConstraintViolationException: Mappa (Key: Property Path, Value: ConstraintDescriptor)

MethodArgumentNotValidException: Mappa (Key: Object Name, Value: String)

Nodo	Valore
AuditEvent	
Id	"input_validation_fail"
EventDesc	"Errore nei dati di input"
EventLevel	2
EventType	1
PrivResource	"N"
Status	"failure"
EventData	
Reason	Exception Message
AuditUser	
PrivUser	"N"
AuditTransaction	
Priority	108

5.2.4 Cifratura dei log

Dalle versioni 2025.02.01 e 2024.07.01 di Springline 2 è disponibile un message converter:

it.ariaspa.springline2.logging.logback. EncryptMsgConverter

Esso permette la cifratura dei messaggi di log tramite una chiave pubblica presente in un keystore. Di seguito le properties che devono essere utilizzare per la configurazione del keystore:

spl:

logging:

encrypt:

keyStore:

location: file:certs/keystore.jks

password: storepass

type: JKS

alias: aliaskey

E' importante che le properties siano inserite nel file bootstrap.yml

Per usarlo è necessario referenziarlo nella configurazione di logback:

```
<configuration>
  <conversionRule conversionWord="encryptmsg"
    converterClass="it.ariaspa.springline2.logging.logback.EncryptMsgConverter" />

  <springProperty name="ksLocation" source="spl.logging.encrypt.keyStore.location"/>
  <springProperty name="ksPassword" source="spl.logging.encrypt.keyStore.password"/>
  <springProperty name="ksType" source="spl.logging.encrypt.keyStore.type"/>
  <springProperty name="kAlias" source="spl.logging.encrypt.keystore.alias"/>

  ..... *
  ..... *

  <appender name="MON" class="ch.qos.logback.core.FileAppender">
    <file>${MON_FILE}</file>
    <append>true</append>
    <encoder>
      <pattern>%encryptmsg${ksLocation},${ksPassword},${ksType},${kAlias}}%n</pattern>
    </encoder>
  </appender>

  ..... *
  ..... *

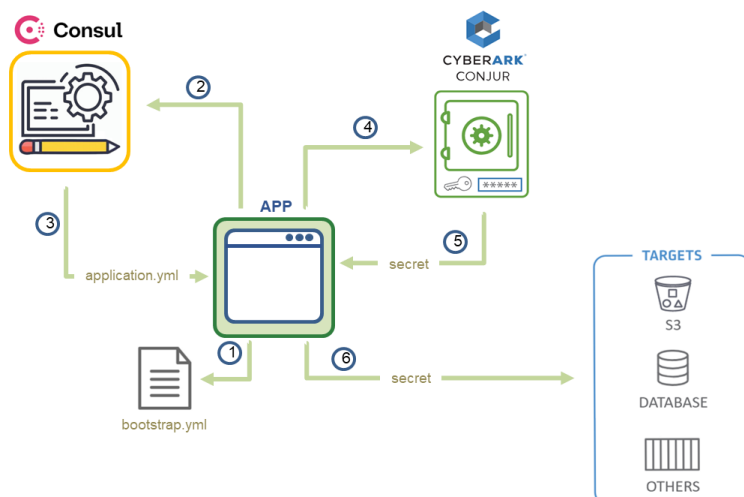
</configuration>
```

Ogni riga dei file di log sarà formata da due campi separati dal carattere | (pipe)

1. Il primo contiene in formato Base64 la chiave AES utilizzata per la cifratura del messaggio di log, cifrata con la chiave pubblica contenuta nel keystore
2. Il secondo contiene in formato Base64 il messaggio di log cifrato con la chiave AES contenuta nel primo campo.

5.3 SpringLine 2 – Config

Nelle architetture software contemporanee, la centralizzazione delle configurazioni è divenuta una pratica consolidata. SpringLine 2, allineandosi a tale tendenza, ha integrato il modulo "Spring Cloud Config", adottando non solo le capacità offerte da Spring Cloud ma arricchendole con funzionalità aggiuntive. Dalle versioni 2025.02.01 e 2024.07.01 è stato introdotto il modulo springlin2-config che consente di centralizzare la configurazione delle applicazioni Spring Boot utilizzando HashiCorp Consul come key/value store e di gestire in modo sicuro i segreti (es. password, token) tramite CyberArk Conjur.



HashiCorp Consul si presenta come una soluzione completa per il service networking, progettata per facilitare la connessione e la protezione di servizi distribuiti su diverse piattaforme e ambienti cloud. Tra le sue funzionalità principali spiccano il service discovery, la gestione del service mesh e un versatile key/value store. Oltre a queste, Consul offre capacità per la configurazione distribuita, la gestione di eventi distribuiti, il locking e le sessioni distribuite, supportando architetture multi-datacenter e fornendo un'interfaccia utente intuitiva. La sua architettura poliedrica lo rende una scelta solida per la gestione di microservizi, dove la configurazione distribuita rappresenta un aspetto cruciale.

Spring Cloud Consul Config emerge come un'alternativa efficace a Spring Cloud Config Server e Client per la gestione della configurazione all'interno di applicazioni Spring Boot. Questo modulo si integra profondamente con il ciclo di vita dell'applicazione, caricando i dati di configurazione nell'Environment di Spring durante la fase di "bootstrap". Questo processo avviene prima che l'applicazione principale sia completamente inizializzata, garantendo che le impostazioni di configurazione siano disponibili fin dalle prime fasi dell'avvio.

L'adozione di Consul per la configurazione distribuita apporta numerosi vantaggi agli sviluppatori di microservizi. In primo luogo, consente la centralizzazione della configurazione per tutti i servizi, semplificando la gestione e garantendo la coerenza. In secondo luogo, supporta gli aggiornamenti dinamici della configurazione senza la necessità di riavviare le applicazioni, un aspetto

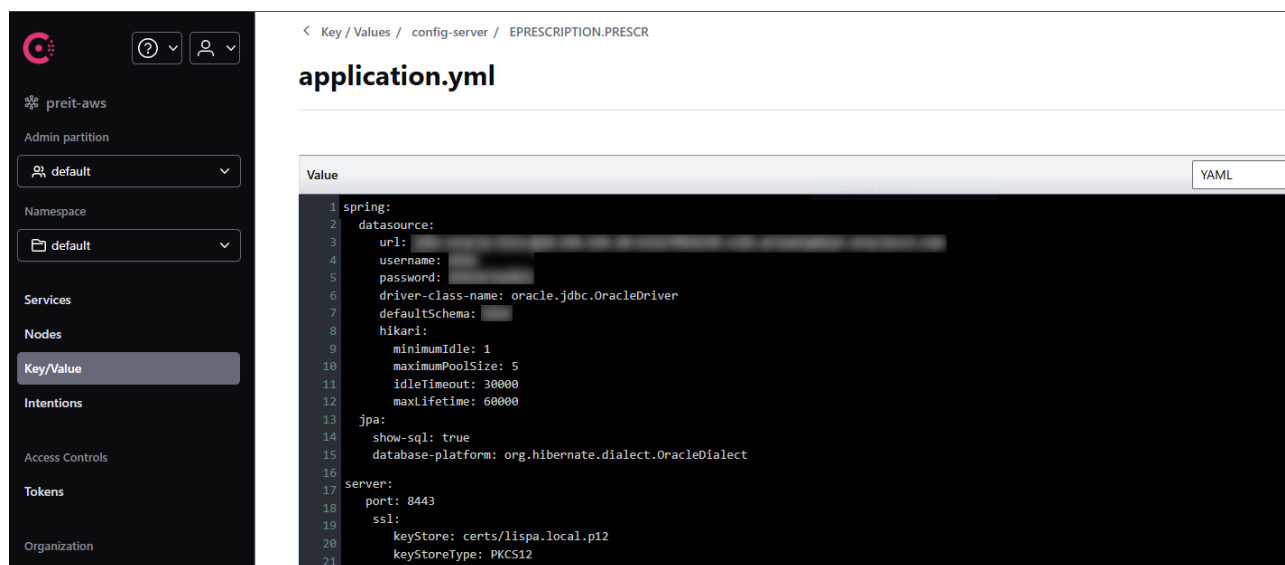
fondamentale per ridurre i tempi di inattività e consentire modifiche più rapide in ambienti di produzione. Questa capacità di aggiornamento in tempo reale è resa possibile da meccanismi di "watch" o notifica da parte di Consul, a cui l'applicazione Spring Boot può reagire

Il cuore della configurazione distribuita con Consul risiede nel suo Key/Value Store, un meccanismo che permette di memorizzare configurazioni e altri metadati in modo centralizzato. Per organizzare i dati di configurazione, Spring Line 2 adotta una struttura di chiavi predefinita. Il percorso base è /config-server. All'interno di questa cartella, viene creata dinamicamente una istanza di PropertySource basata sul nome dell'applicazione. La configurazione viene memorizzata in un intero blocco di proprietà in formato YAML all'interno di una singola chiave in Consul con nome predefinito application.yml.

Per impostazione predefinita, la configurazione viene archiviata nella cartella **config-server/<nome-applicazione>/application.yml**

Ad esempio, un'applicazione con nome "EPRESCRIPTION.PRESCR" avrà la seguente struttura di key e folder in Consul:

```
config-server\           # folder
    EPRESCRIPTION.PRESCR\ # folder
        application.yml # key
```



ATTENZIONE

Il termine “application” è una parola chiave riservata e non può essere usato come nome applicazione

La configurazione viene attualmente letta all'avvio dell'applicazione. L'invio di un http POST a /refresh causerà il ricaricamento della configurazione. Config Watch rileverà automaticamente le modifiche e ricaricherà il contesto dell'applicazione.

Per consentire a un'applicazione Spring Boot di comunicare con un'istanza di HashiCorp Consul, è necessario configurare i dettagli della connessione nel file bootstrap.yml. La ragione principale è che bootstrap.yml viene caricato in una fase precedente del ciclo di vita dell'applicazione rispetto a application.properties. Questo permette di configurare i componenti di Spring Cloud, come Consul Config, prima che l'applicazione principale inizi il suo processo di avvio. Di seguito un esempio di configurazione:

```
spring:
  application:
    name: <nome applicazione> # Nome dell'applicazione

  cloud:
    consul:
      enabled: true
      host: <consul endpoint> # Indirizzo del server Consul
      port: <consul port>    # Porta di Consul
      scheme: HTTPS         # Protocollo di connessione
      config:
        acl-token: <consul token> # Token ACL per accedere a Consul

  spl:
    conjur:
      enabled: true
      appliance-url: <conjur endpoint> # Endpoint del server Conjur
      account: <conjur account>        # Nome dell'account Conjur
      authn:
        login: <conjur login>          # Login per l'autenticazione
        api-key: <conjur apikey>       # API key per l'autenticazione
      tls-ca-path: <conjur ca certificate> # Percorso al certificato CA per TLS
```

La libreria carica automaticamente il file application.yml dal key/value store di Consul utilizzando il prefisso e la chiave configurati in spring.cloud.consul.config. Le configurazioni saranno disponibili come normali proprietà di Spring.

Dalle versioni 2025.03.01 e 2024.08.01 è stata introdotta una cache locale della configurazione che viene utilizzata qualora vi siano problemi di comunicazione con il configuration server. La cache locale è abilitata di default e viene salvata nel path di esecuzione dell'applicazione. E' possibile modificare questo default per mezzo delle properties:

```
spl.consul.cache.path=  
spl.consul.cache.enabled=true
```

Proprietà risolte da Conjur

Per definire una proprietà che verrà risolta da CyberArk Conjur, utilizzare il placeholder **conjur:** nel file `application.yml` dell'applicazione. Esempio:

```
spring:  
  datasource:  
    url: jdbc:mysql://<host>:<port>/<db_name>  
    username: my_db_user  
    password: conjur:/db/database1/password
```

In questo caso, la libreria recupererà il segreto `db/database1/password` da Conjur e lo userà come valore per `spring.datasource.password`.

NB. Sia `acl-token` di Consul che `api-key` di Conjur posso essere inseriti nel file in forma cifrata tramite la cifratura offerta da `spring-cloud-config`.

Accesso alle proprietà

Le proprietà caricate da Consul o risolte da Conjur possono essere utilizzate come normali proprietà di Spring Boot.

Lettura con `@Value`

```
@Value("${spring.datasource.url}")  
private String dataSourceUrl;
```

Lettura con `Environment.getProperty()`

```
@Autowired  
private Environment environment;  
  
public void printDataSourceUrl() {  
    String dataSourceUrl = environment.getProperty("spring.datasource.url");  
    System.out.println("Datasource URL: " + dataSourceUrl);  
}
```

Gestione dei segreti con rotazione

Per i segreti gestiti da Conjur, che possono essere ruotati dinamicamente, è fondamentale utilizzare `Environment.getProperty()`. Questo metodo garantisce che venga sempre letto il valore aggiornato del segreto, evitando eventuali cache o riferimenti obsoleti.

Aggiornamenti della Configurazione in Tempo Reale

Una delle funzionalità più potenti di Spring Cloud Consul Config è la capacità di gestire gli aggiornamenti della configurazione in tempo reale senza richiedere il riavvio dell'applicazione. Questo è possibile grazie al Config Watch e all'endpoint actuator /refresh.

Il Config Watch è un meccanismo che monitora un prefisso di chiave specifico in Consul per rilevare eventuali modifiche. Questo avviene tramite una chiamata bloccante all'API HTTP di Consul. Se vengono rilevati dati di configurazione modificati che sono rilevanti per l'applicazione corrente, viene pubblicato un Refresh Event. Questo evento è equivalente a chiamare l'endpoint actuator /refresh.

La frequenza con cui il Config Watch interroga Consul può essere configurata tramite la proprietà `spring.cloud.consul.config.watch.delay`, il cui valore predefinito è di 1000 millisecondi. È anche possibile disabilitare completamente il Config Watch impostando `spring.cloud.consul.config.watch.enabled` su `false`.

Oltre all'aggiornamento automatico fornito dal Config Watch, è possibile attivare manualmente un ricaricamento della configurazione inviando una richiesta HTTP POST all'endpoint `/actuator/refresh`. Per utilizzare questo endpoint, è necessario includere la dipendenza `spring-boot-starter-actuator` nel progetto. Questo endpoint offre un modo on-demand per aggiornare la configurazione, utile in scenari specifici o durante le fasi di sviluppo e test.

Per far sì che i bean Spring reagiscano agli aggiornamenti della configurazione senza richiedere un riavvio, è necessario utilizzare l'annotazione `@RefreshScope`. Quando viene pubblicato un Refresh Event, tutti i bean annotati con `@RefreshScope` vengono ricreati, garantendo che utilizzino i nuovi valori di configurazione. Questa annotazione è particolarmente utile per i bean che iniettano proprietà di configurazione tramite `@Value` o che fanno parte di classi annotate con `@ConfigurationProperties`.

Ad esempio:

```
@RefreshScope
@Component
public class MyService {
    @Value("${my.property}")
    private String myProperty;
```

```
//...  
}
```

In questo esempio, se il valore della proprietà `my.property` viene modificato in Consul, il bean `MyService` verrà ricreato e il campo `myProperty` conterrà il nuovo valore.

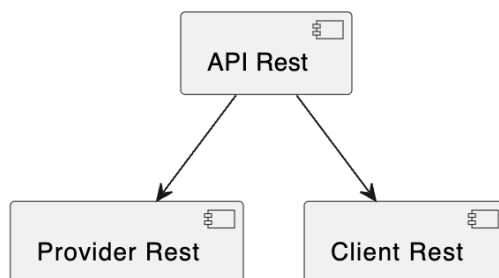
Per ulteriori informazioni e configurazioni fare riferimento alla documentazione di Spring Cloud Consul: <https://docs.spring.io/spring-cloud-consul/reference/config.html>

Si ricorda che per utilizzare queste funzionalità è necessario l'inserimento nel `pom.xml` della seguente dipendenza:

```
<!-- SPRINGLINE2 CONFIG: per l'uso di Cconsul e Conjur. Dalle versioni 2025.02.01 e  
2024.02.01 -->  
<dependency>  
  <groupId>it.ariaspa.springline2</groupId>  
  <artifactId>springline2-config</artifactId>  
  <version>${spring-line.version}</version>  
</dependency>
```

* Per le versioni vedere l'appendice A

5.4 SpringLine 2 – API Rest



Nell'odierno panorama informatico, le architetture software si stanno evolvendo rapidamente, e la necessità di esporre o interagire con servizi esterni attraverso API REST è diventata una pratica comune.

Caratterizzate dalla loro essenza stateless, dalla capacità di aderire agli standard web consolidati e dalla versatilità nella manipolazione di vari formati di dati, le API REST si sono affermate come la soluzione prediletta per un'ampia gamma di applicazioni contemporanee.

In questo contesto, Spring, attraverso il suo modulo '**spring-web**', ha offerto una piattaforma solida e adattabile, permettendo agli sviluppatori di costruire servizi con efficienza e resilienza.

SpringLine 2, riconoscendo l'importanza di questa interazione, ha sviluppato una soluzione che estende il comportamento di default di Spring sia come Provider Rest che Client Rest.

5.4.1 Provider Rest

SpringLine 2, pur valorizzando le robuste capacità di 'spring-web', ha esteso ulteriormente le sue funzionalità. L'obiettivo principale di SpringLine 2 è personalizzare la modalità con cui le API Rest rispondono al chiamante, garantendo una comunicazione in linea con attuali linee guida Aria S.p.A.

Laddove Spring mette a disposizione l'oggetto `ResponseEntity` per orchestrare le risposte HTTP, SpringLine 2 va oltre, introducendo **RestResponse** e **RestResponseBuilder**.

Con SpringLine 2, gli stati HTTP vengono intuitivamente suddivisi in:

- Positivi: status HTTP 20x e 30x;
- Negativi: status HTTP 40x e 50x.

In relazione a questa suddivisione:

- `RestResponse` facilita la creazione sia di risposte positive che negative.
- `RestResponseBuilder`, invece, è focalizzato solo sulla generazione di risposte positive, offrendo la flessibilità di personalizzare gli headers.

Per comporre la response, entrambe le componenti utilizzano metodi di classe specifici, uno per ogni status HTTP (in linea con lo standard internazionale definito dall'Internet Engineering Task Force (IETF) RFC 7231).

Per quanto riguarda le risposte positive, non vi è distinzione tra `RestResponse` e `RestResponseBuilder`. Il loro comportamento, infatti, si riconduce allo stesso comportamento adottato da `ResponseEntity`. Tuttavia, la differenza diventa evidente in presenza di eventi eccezionali. In tali scenari, l'evento eccezionale viene propagato attraverso lo stack delle chiamate fino al Controller. A questo punto, spetta allo sviluppatore:

- gestirlo manualmente utilizzando uno dei metodi di classe di `RestResponse`. Dovranno essere fornite informazioni aggiuntive come URI, codice, titolo, dettagli e sub-errors.
- lanciare una delle eccezioni specifiche fornite da SpringLine 2, corrispondenti allo status HTTP appropriato (tipicamente `HttpStatus` della famiglia 40x). Queste eccezioni vengono intercettate dal `GlobalExceptionHandler` di SpringLine 2, che formula la risposta al chiamante sollevando lo sviluppatore dal doversi occupare di tracciare in modo opportuno tale evento (scrittura degli Audit Log). Il `GlobalExceptionHandler` si occupa di gestire anche eventi eccezionali più specifici: `MethodArgumentTypeMismatchException` e `ConstraintViolationException`, offrendo risposte mirate e dettagliate, arricchendo l'esperienza per il chiamante.

Di seguito alcuni esempi Java che mostrano come usare `RestResponse` per la generazione di risposte positive e negative oppure la propagazione di eccezioni:

```
@GetMapping("/cool-cars")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.OK)
@Operation(description = "liste delle macchine cool",
    responses = { @ApiResponse(
        description = "Successful Operation",
        responseCode = "200",
        content = @Content(mediaType = "application/json"))
    })
public ResponseEntity<Collection<Car>> coolCars() {
    Context context = ContextHolder.currentContext();

    if ("PRIVILEGE_COOL".equals(user.getClaim("privilege"))) {
        Collection<Car> cars = carService.coolCars();
        return RestResponse.ok(cars);
    }

    throw new ForbiddenException(null, 1,
        "Privilegi non sufficienti", "Utente senza privilegi per
        leggere l'elenco delle cool cars");
}
```

5.4.2 Client Rest

SpringLine 2 propone un client REST avanzato e completamente configurabile. La componente può essere attivata o disattivata attraverso la proprietà di configurazione descritta nella seguente tabella

Proprietà	Valore
spl.client-rest.autoconfiguration.enabled	True/False Valore di Default: True

La gestione del client REST customizzato è resa trasparente allo sviluppatore, il quale deve semplicemente effettuare un constructor injection della classe **RestTemplateBuilder** di **spring-web** al fine di recuperare l'istanza di **RestTemplate**.

Di seguito un esempio Java:

```
@Service
public class ExampleService {

    @Value("${url}")
    private String url;

    private final RestTemplate restTemplate;

    public ExampleService(final RestTemplateBuilder
                           restTemplateBuilder) {
        this.restTemplate = restTemplateBuilder.build();
    }

    public String method() {
        return restTemplate.getForEntity(url,
                                         String.class).getBody();
    }
}
```

SpringLine 2, operando dietro le quinte, personalizza questa istanza integrando:

- i logger "MON" e "MON-APP"
- la propagazione delle informazioni dell'utente loggato verso sistemi terzi che dipendono da SpringLine 1 o SpringLine 2.

Una volta ottenuta l'istanza di **RestTemplate**, lo sviluppatore può utilizzarla seguendo le convenzioni e le linee guida standard di Spring.

Di seguito, la tabella di riferimento che elenca le properties che consentono la personalizzazione sia dei log di monitoraggio che della propagazione.

Dalle versioni **2023.05.01** e **2024.04.01** è stata inserita la gestione dei retries che possono essere abilitate tramite impostazioni delle seguenti properties:

Proprietà	Valore
spl.client-rest.retry.enabled	true/false Valore di default false
spl.client-rest.retry.uri-matchers	Valore di default: /** Dominio di valori: String Identifica gli uri per i quali verrà effettuato il retry. E' possibile definire l'elenco degli end point separati da “,”
spl.client-rest.retry.excluded-uri-matchers	Valore di Default: Dominio di valori: String Identifica gli uri per i quali il retry non verrà effettuato.E' possibile definire l'elenco degli end point separati da “,”
spl.client-rest.retry.max-retries	Valore di Default: 3 Dominio di valori: Number Indica il numero di retries che verranno eseguiti.
spl.client-rest.retry.with-timeout	Valore di Default: 0 Dominio di valori: Number >= 0 Riprova fino allo scadere del timeout,in millisecondi, dal tentativo iniziale. Se 0 non viene impostato nessun valore di timeout.
spl.client-rest.retry.fixed-backoff	Valore di Default: 0 Dominio di valori: Number >=0 Esegue ciascun tentativo dopo un periodo di tempo fisso in millisecondi.

5.4.3 MON

Proprietà	Valore
spl.client-rest.logging.mon.enabled	True/False Valore di default True
spl.client-rest.logging.mon.uri-matchers	Valore di default: /** Dominio dei valori: String Identifica gli uri per i quali verrà effettuato il log. E' possibile definire l'elenco degli end point separati da “,”
spl.client-rest.logging.mon.excluded-uri-matchers	Valore di Default: Stringa Vuota Dominio dei valori: String Identifica gli end point per i quali il log non sarà tracciato. E' possibile definire l'elenco degli end point separati da “,”

5.4.4 MON-APP

Proprietà	Valore
spl.client-rest.logging.app.enabled	True/False Valore di default False
spl.client-rest.logging.app.uri-matchers	Valore di Default: /** Dominio dei valori: String Identifica gli end point per i quali sarà tracciato il log E' possibile definire l'elenco degli end point separate da “,”

Proprietà	Valore
spl.client-rest.logging.app.excluded-uri-matchers	Valore di Default: String Empty Dominio dei valori: String Identifica gli end point per i quali il log non sarà tracciato. E' possibile definire l'elenco degli uri separati da “,”
spl.client-rest.logging.app.excluded-content-types	Valore di Default: Empty Dominio dei valori: String Identifica l'elenco dei Content-types per i quali il log non sarà tracciato. Ogni elemento è separato da “,”
spl.client-rest.logging.app.body-max-length	Default Value: 1048576 Dominio dei valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

5.4.5 Propagation SpringLine 2

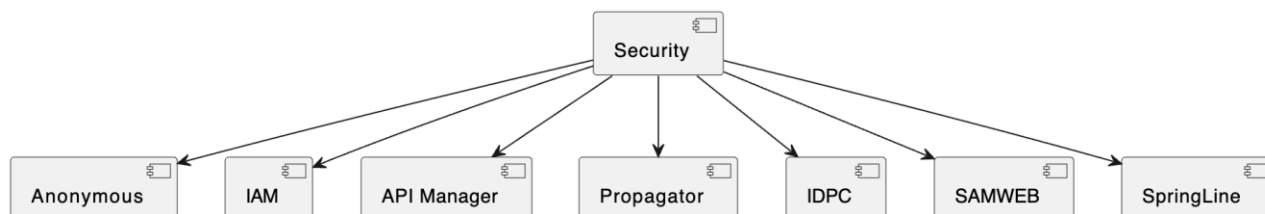
Proprietà	Valore
spl.client-rest.propagation.springline2.enabled	True/False Valore di Default: False
spl.client-rest.propagation.springline2.uri-matchers	Valore di Default: Stringa Vuota Dominio dei valori: String Identifica gli end point per i quali avverrà la propagazione. E' possibile definire l'elenco degli end point separate da “,”
spl.client-rest.propagation.springline2.excluded-uri-matchers	Valore di Default: String Empty Dominio dei valori: String Identifica gli end point per i quali non avverrà la propagazione. E' possibile definire l'elenco degli uri separati da “,”

5.4.6 Propagation SpringLine

Proprietà	Valore
spl.client-rest.propagation.springline.enabled	True/False Valore di Default: False
spl.client-rest.propagation.springline.uri-matchers	Valore di Default: Stringa Vuota Dominio dei valori: String Identifica gli end point per i quali avverrà la propagazione E' possibile definire l'elenco degli end point separate da “,”
spl.client-rest.propagation.springline.excluded-uri-matchers	Valore di Default: String Empty Dominio dei valori: String Identifica gli end point per i quali non avverrà la propagazione. E' possibile definire l'elenco degli uri separati da “,”
spl.client-rest.propagation.springline.header.username	x-springline-username
spl.client-rest.propagation.springline.header.authchannel	x-springline-authchannel

Proprietà	Valore
spl.client-rest.propagation.springline.header.authmethod	x-springline-authmethod
spl.client-rest.propagation.springline.header.authstrength	x-springline-authstrength
spl.client-rest.propagation.springline.header.role	x-springline-role
spl.client-rest.propagation.springline.header.firstname	x-springline-firstname
spl.client-rest.propagation.springline.header.lastname	x-springline-lastname
spl.client-rest.propagation.springline.header.realvirtual	x-springline-realvirtual
spl.client-rest.propagation.springline.header.fiscalcode	x-springline-fiscalcode
spl.client-rest.propagation.springline.header.email	x-springline-email
spl.client-rest.propagation.springline.header.credential	x-springline-credential
spl.client-rest.propagation.springline.header.appname	x-springline-appname

5.5 SpringLine 2 – Security



Il modulo di Autenticazione di SpringLine 2 è stato progettato sulle solide fondamenta offerte da Spring Security. SpringLine 2 non si limita a replicare le funzionalità di base, ma le estende e le personalizza per integrare diversi sistemi di autenticazione, garantendo al contempo la conformità agli elevati standard di sicurezza di Aria S.p.A.

È doveroso premettere che, per impostazione predefinita, Spring Security crea automaticamente una password all'avvio dell'applicativo quando si utilizza l'autenticazione di base. Tuttavia, fare affidamento su una password generata automaticamente non è consigliabile in un contesto di produzione. Questa caratteristica è pensata principalmente per ambienti di sviluppo e test. SpringLine 2 offre la possibilità di disattivare questa funzionalità di Spring Security, fornendo una property specifica da configurare nel file delle impostazioni dell'applicazione.

Proprietà	
spl.security.generated-dev-password.enabled	True/False Valore di Default: False

Gli Identity Provider con cui SpringLine 2 si integra, sono:

Anonymous: Pur introducendo la figura dell'utente anonimo, Spring Line 2 garantisce che l'accesso e l'interazione di tali utenti siano gestiti in modo sicuro e controllato, in conformità con gli standard di Aria S.p.A.;

IAM: Sistema di Single Sign-On adottato in collaborazione con enti regionali. Basato su CA, propaga le informazioni dell'utente che ha effettuato l'accesso, garantendo un'esperienza di login unificata e sicura;

API Manager: L'identity Provider consente a SpringLine 2 di identificare l'utente connesso al sistema e di acquisirne le informazioni propagate, assicurando un'interazione sicura e conforme.

Propagator: sistema progettato per raccogliere le informazioni dell'utente da un'altra applicazione che incapsula SpringLine 2, assicurando l'integrità e la sicurezza dei dati durante la trasmissione;

IDPC: sistema di autenticazione che si basa sull'identità digitale del cittadino;

SAMWEB: sistema di identificazione e autorizzazione per gli operatori sanitari del sistema SISS;

SpringLine: sistema di autenticazione che consente ad applicativi SpringLine 2 di integrarsi con applicativi SpringLine 1;

In SpringLine 2, il JWT (**JSON Web Token**) non viene adoperato come principale meccanismo di autenticazione, ma piuttosto come veicolo per arricchire i dettagli associati all'utente. In situazioni tipiche dove un portale centrale si interfaccia con varie applicazioni verticali, dopo che l'utente si autentica sul portale e decide di interagire con un'applicazione verticale specifica, il JWT entra in gioco. Esso permette di estendere e integrare le informazioni dell'utente, fornendo dettagli aggiuntivi legati all'applicazione verticale selezionata.

In sintesi, SpringLine 2 rappresenta un'evoluzione nell'ambito dell'autenticazione, offrendo soluzioni avanzate e conformi agli standard di Aria S.p.A., garantendo agli sviluppatori flessibilità nella scelta del sistema di autenticazione, mantenendo un elevato livello di sicurezza.

5.5.1 Configurazione Base SpringLine 2 - Security

SpringLine 2 facilita lo sviluppatore offrendo una configurazione di sicurezza di base.

```
@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain actuatorFilterChain(final HttpSecurity http) throws Exception {
        return http.securityMatcher("/actuator/**")
            .authorizeHttpRequests(authorizeHttpRequests ->
                authorizeHttpRequests.anyRequest().permitAll())
            .build();
    }

    @Bean
    SecurityFilterChain errorFilterChain(final HttpSecurity http) throws Exception {
        return http.securityMatcher("/error")
            .authorizeHttpRequests(authorizeHttpRequests ->
                authorizeHttpRequests.dispatcherTypeMatchers
                    (DispatcherType.ERROR).permitAll()).build();
    }

    @Bean
    SecurityFilterChain protectedFilterChain(final HttpSecurity http) throws Exception {
        http.securityMatcher("/**")
            .authorizeHttpRequests(authorizeHttpRequests ->
                authorizeHttpRequests.anyRequest().authenticated())
            .logout(logout -> logout.disable())
            .httpBasic(httpBasic -> httpBasic.disable())
            .sessionManagement(sessionManagement ->
                sessionManagement.sessionCreationPolicy
                    (SessionCreationPolicy.STATELESS))
            .csrf(csrf -> csrf.disable());

        return addDefaultFilter(http).build();
    }
}
```

Il Bean **actuatorFilterChain** consente l'accesso senza autenticazione al percorso `/actuator/**`, spesso utilizzato per endpoint di monitoraggio e gestione.

Il Bean **errorFilterChain** è configurato per gestire il percorso `/error`. Il filtro si occupa delle richieste correlate agli errori, permettendo l'accesso senza autenticazione per personalizzare la gestione degli errori.

Il Bean **protectedFilterChain** costituisce il cuore della configurazione di sicurezza, applicato a tutti i percorsi (**/****). Richiede l'autenticazione per tutte le richieste, disabilita la gestione delle sessioni, l'autenticazione HTTP di base, il logout e fornisce protezione CSRF.

E' necessario sottolineare che la catena di filtri risultante include i filtri predefiniti di SpringLine 2 aggiunti tramite `addDefaultFilter(http)`.

5.5.2 Configurazione Custom SpringLine 2 - Security

SpringLine 2 offre una flessibilità estesa per la personalizzazione della sicurezza. Questo è particolarmente rilevante quando la configurazione di sicurezza di default o i sistemi di autenticazione con cui SpringLine 2 si integra, non sono sufficienti o non aderiscono allo specifico requisito. Nel seguente esempio, viene illustrato come aggiungere l'autenticazione di base per la comunicazione Application to Application:

```
@EnableWebSecurity
@Configuration
public class SecurityConfiguration {

    @Autowired
    private SecurityFilterConfiguration securityFilter;

    @Bean
    /* default */ SecurityFilterChain basicFilterChain(final HttpSecurity http)
                                                    throws Exception {

        http.securityMatcher("/api/**")
            .authorizeHttpRequests(authorizeHttpRequests ->
                authorizeHttpRequests.anyRequest().authenticated())
            .logout(logout -> logout.disable())
            .httpBasic(Customizer.withDefaults())
            .sessionManagement(sessionManagement ->
                sessionManagement.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .csrf(csrf -> csrf.disable());
        securityFilter.addDefaultFilter(http)
            .addFilterAfter(new AuthBasicFilter(),
                BasicAuthenticationFilter.class);

        return http.build();
    }

    public class AuthBasicFilter extends OncePerRequestFilter {

        @Override
        protected boolean shouldNotFilter(final HttpServletRequest request) {
            final Authentication authentication =
                SecurityContextHolder.getContext().getAuthentication();

            return authentication == null ||
                !(authentication.getPrincipal() instanceof UserDetails);
        }

        @Override
        protected void doFilterInternal(final HttpServletRequest request,
                                         final HttpServletResponse response,
                                         final FilterChain chain) throws
```

```

        ServletException, IOException {
    final Object principal = SecurityContextHolder.getContext()
        .getAuthentication().getPrincipal();

    final User user = new User();
    user.setUsername(((UserDetails)principal).getUsername());
    user.setStrength("0");
    user.setCircuit("R");
    user.setChannel("BASIC");
    ContextHolder.currentContext().setUser(user);
    chain.doFilter(request, response);
}
}
}

```

Il Bean **basicFilterChain** definisce una configurazione di sicurezza personalizzata per il percorso `/api/**`, richiedendo l'autenticazione di base. Si noti l'aggiunta di un filtro personalizzato **AuthBasicFilter** che viene eseguito dopo il **BasicAuthenticationFilter**. Questo filtro personalizzato è fondamentale per integrare l'autenticazione di base e impostare correttamente l'oggetto **User** nel contesto di SpringLine 2.

Un punto cruciale da sottolineare è l'importanza di aggiungere SEMPRE i filtri di SpringLine 2 ogni qualvolta si opta per la customizzazione del requisito di sicurezza. Per farlo è necessario dapprima effettuare l'inject di **SecurityFilterConfiguration** e poi richiamare il metodo `addDefaultFilter(http)` sull'istanza iniettata.

5.5.3 Properties User

Di seguito sono descritte le **properties**² utente valorizzate rispetto al sistema di autenticazione.

E' necessario osservare che, laddove la nomenclatura degli headers dovesse differire rispetto a quanto riportato per ogni sistema di autenticazione, è necessario applicare tale variazione nelle impostazioni applicative come riportato nel paragrafo 4.2

5.5.3.1 Anonymous

Proprietà	Value/Header
Username	"ANONYMOUS"
Strength	"0"
Circuit	"R"
Channel	"ANONYMOUS"

5.5.3.2 IAM

Le proprietà Utente sono disponibili solo se presente e valorizzato il seguente headers:

² Senza l'uso dei doppi apici, il valore indicato può variare in relazione all'intestazione associata, mentre all'interno dei doppi apici sono inseriti i valori costanti.

- http_codicefiscale

Proprietà	Value/Header
Username	http_codicefiscale
FiscalNumber	http_codicefiscale
Name	http_nome
FamilyName	http_cognome
Email	http_email
Strength	"65"
Circuit	"R"
SerialNumber	http_matricola
Channel	"IAM"

5.5.3.3 API Manager

Le proprietà Utente sono disponibili solo se presente e valorizzato il seguente headers:

- x-jwt-assertion

La tipologia di utente è definita nel nodo `http://wso2.org/claims/usertype`

5.5.3.4 Utente Applicativo (Cittadino/Operatore)

Proprietà	Value/Header
Username	<code>http://wso2.org/claims/username:</code> • se il valore di <code>http://wso2.org/claims/usertype</code> è "APPLICATION_USER" • se non è presente il nodo <code>http://wso2.org/claims/reale</code> • se non è presente e non valorizzato l'header <code>activcred</code> <code>UserId</code> (presente nell'header <code>activcred</code>): • se il valore di <code>http://wso2.org/claims/usertype</code> è "APPLICATION_USER" • se non è presente il nodo <code>http://wso2.org/claims/reale</code> • se è presente e valorizzato l'header <code>activcred</code>
FiscalNumber	<code>http://wso2.org/claims/username:</code> • se il valore di <code>http://wso2.org/claims/usertype</code> è "APPLICATION_USER"
Name	<code>http://wso2.org/claims/fullname:</code> • se il valore di <code>http://wso2.org/claims/usertype</code> è "APPLICATION_USER" <code>FirstName</code> (presente nell'header <code>activcred</code>): • se il valore di <code>http://wso2.org/claims/usertype</code> è "APPLICATION_USER" • se non è presente il nodo <code>http://wso2.org/claims/reale</code> • se è presente e valorizzato l'header <code>activcred</code>

Proprietà	Value/Header
FamilyName	http://wso2.org/claims/lastname: - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" LastName (presente nell'header activcred): - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se è presente e valorizzato l'header activcred
Circuit	"R": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale - se il valore del nodo http://wso2.org/claims/reale è "TRUE" oppure - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se non presente o non valorizzato l'header activcred - se è presente e valorizzato l'header activcred ed il valore è < 50 "V": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale - se il valore del nodo http://wso2.org/claims/reale è "FALSE" oppure - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se è presente e valorizzato l'header activcred
Role	"49": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale - se il valore del nodo http://wso2.org/claims/reale è "TRUE" "99": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale - se il valore del nodo http://wso2.org/claims/reale è "FALSE" Role (presente nell'header activcred): - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se è presente e valorizzato l'header activcred
Method	"CITTADINI": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale "OPERATORI": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se non è presente e non valorizzato l'header activcred AuthType - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se è presente e valorizzato l'header activcred

Proprietà	Value/Header
Strength	"0": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se AuthType dell'header activcred non è presente o non valorizzato "60" - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se AuthType dell'header activcred è presente - se il valore AuthType dell'header activcred è "BASIC" "70": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/reale oppure - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se non è presente e valorizzato l'header activcred "90" - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se AuthType dell'header activcred è presente - se il valore AuthType dell'header activcred è "OTP" "100": - se il valore di http://wso2.org/claims/usertype è "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/reale - se AuthType dell'header activcred è presente - se il valore AuthType dell'header activcred è "SCOP", "SCOPIDPC", "CRS"
Organization	Concatenazione di L1 L2 L3. Il carattere di separazione tra le 3 stringhe è "-"
Channel	"APIMANAGER"

5.5.3.5 Servizio

Proprietà	Value/Header
Username	http://wso2.org/claims/organization
Circuit	"R": - se il valore di http://wso2.org/claims/usertype è diverso da "APPLICATION_USER" - se non è presente il nodo http://wso2.org/claims/apicontext oppure è presente ma non valorizzato "V" - se il valore di http://wso2.org/claims/usertype è diverso da "APPLICATION_USER" - se è presente il nodo http://wso2.org/claims/apicontext ed il valore è "SERVIZI.RL" - se è presente il nodo http://wso2.org/claims/keytype ed il valore è "SANDOX"
Method	"SERVIZI"
Channel	"APIMANAGER"

5.5.3.6 Propagator

Proprietà	Value/Header
User	Le informazioni dell'utente dipendono dal sistema oggetto di integrazione.

5.5.3.7 IDPC

Le proprietà Utente sono disponibili solo se presenti e valorizzati almeno uno dei due seguenti headers:

- saml_attribute_nomeutente
- saml_attribute_codicefiscale

Se l'header saml_attribute_nomeutente non è disponibile, lo stesso sarà valorizzato con l'header saml_attribute_codicefiscale

Proprietà	Value/Header
Username	saml_attribute_nomeutente
FiscalNumber	saml_attribute_codicefiscale
Name	saml_attribute_nome
FamilyName	saml_attribute_cognome
Circuit	"R" • se presente l'header saml_attribute_cns_carta_reale. "V" altrimenti
Role	"49" • se presente l'header saml_attribute_cns_carta_reale. "99" altrimenti.
Email	saml_attribute_emailaddress
Method	saml_attribute_tipoautenticazione
Strength	saml_attribute_livelloverifica
Gender	saml_attribute_sesso
DateOfBirth	saml_attribute_datanascita
PlaceOfBirth	saml_attribute_luogonascita
CountryOfBirth	saml_attribute_provincianascita
NationOfBirth	saml_attribute_statonascita
MobilePhone	saml_attribute_cellulare
CompanyName	saml_attribute_ragionesociale

Proprietà	Value/Header
RegisteredOffice	saml_attribute_sedelegale
IvaCode	saml_attribute_partitaiva
IDCard	saml_attribute_docidentita
ExpirationDate	saml_attribute_scaddocidentita
Address	saml_attribute_domiciliofisico
DigitalAddress	saml_attribute_domiciliodigitale
Issuer	saml_attribute_cns_issuer
Subject	saml_attribute_cns_subject
DataSource	saml_attribute_originedatiutente
SafeCredential	saml_attribute_idcomuneregistrazione
PreviousLoginDate	Concatenazione dei seguenti headers: - data_ultimo_login - ora_ultimo_login il carattere di separazione tra i due headers è " " (blank space)
IPAddress	indirizzo_ip_corrente
PreviousIpAddress	indirizzo_ip_precedente
PreviousMethod	saml_attribute_tipo_spid
DomicileStreetAddress	saml_attribute_servizio_ultimo_login
DomicilePostalCode	saml_attribute_capdomiciliofisico
DomicileMunicipality	saml_attribute_comunedomiciliofisico
DomicileProvince	saml_attribute_provinciadomiciliofisico
DomicileNation	saml_attribute_nazionedomiciliofisico
PreviousIdp	saml_attribute_idp_ultimo_login
Organization	organizzazione:organizzazione
SerialNumber	matricola
InstitutionName	enteappartenenza
InstitutionDescription	descrizioneente
EmploymentStatus	statolavorativo

Proprietà	Value/Header
Channel	"IDPC"

5.5.3.8 SAMWEB

Le proprietà Utente sono disponibili solo se presente e valorizzato il seguente header:

- activcred

Proprietà	Value/Header
Assertion	activcred
UID	uid
Token	token
Token SSO DN	token_ssodn
Username	UserId
Name	FirstName
FamilyName	LastName
Role	Role
Circuit	"R" • se Ruolo ha un valore strettamente inferiore a 50 "V" altrimenti
Method	AuthType
Strength	"0": • se AuthType non è presente o non valorizzato "60": • se AuthType è presente ed è valorizzato con "BASIC" "90": • se AuthType è presente ed è valorizzato con "OTP" "100" • se AuthType è presente ed è valorizzato con "SCOP" oppure "SCOPIDPC" oppure "CRS"
TrackId	uid
Organization	Concatenazione di L1 L2 L3. Il carattere di separazione tra le 3 stringhe è "-"
Channel	"SAMWEB"

5.5.3.9 SpringLine

Le proprietà Utente sono disponibili solo se presente e valorizzato il seguente header:

- x-springline-username

Proprietà	Value/Header
Username	x-springline-username
Name	x-springline-firstname
FamilyName	x-springline-lastname
FiscalNumber	x-springline-fiscalcode
Email	x-springline-email
Assertion	x-springline-credential
Circuit	x-springline-realvirtual
Role	x-springline-role
Strength	x-springline-authstrength
Method	x-springline-authmethod
Channel	x-springline-authchannel

5.5.3.10 JWT

Le proprietà Utente sono disponibili solo se presente il JWT e valorizzato il seguente header preceduto dalla stringa "Bearer":

- Authorization

Dalle versioni 2024.02.01 e 2023.03.01 è possibile configurare la ricerca del jwt anche nei cookie.

Si controlla che il claim 'subject' coincida con la proprietà 'username' dell'entità User. In caso di discrepanza, l'autenticazione sarà invalidata. Ogni claim presente nel JWT è inserito nella mappa 'claims' dell'entità User

Proprietà	Value/Header
Claims	Ogni coppia chiave-valore dei claims JWT

Per la generazione dei JWT è disponibile la classe di utilità **JwtToken**. Fornisce metodi per generare e ottenere claim da token JWT. Di seguito è una spiegazione dettagliata dei principali metodi della classe:

generateToken():

Descrizione: Genera un nuovo token JWT senza claims.

Eccezioni: Può generare una JwtException se si verifica un errore nella generazione del token.

Ritorno: Restituisce il token JWT generato.

generateToken(Map<String, Object> claims):

Descrizione: Genera un nuovo token JWT con i claim specificati.

Parametri: Una mappa di claim da includere nel token.

Eccezioni: Può generare una JwtException se si verifica un errore nella generazione del token.

Ritorno: Restituisce il token JWT generato.

generateCookie():

Descrizione: Genera un cookie contenente un nuovo token JWT senza claims.

Eccezioni: Può generare una JwtException se si verifica un errore nella generazione del token.

Ritorno: Restituisce il cookie generato.

generateCookie(Map<String, Object> claims):

Descrizione: Genera un cookie contenente un nuovo token JWT con i claim specificati.

Parametri: Una mappa di claim da includere nel token.

Eccezioni: Può generare una JwtException se si verifica un errore nella generazione del token.

Ritorno: Restituisce il cookie generato.

5.5.4 Authorities

SpringLine 2 fornisce un robusto sistema di autorizzazione attraverso le Authorities, una caratteristica integrata di Spring Security, accessibili attraverso l'oggetto User. Le Authorities consentono una gestione fine delle autorizzazioni sui metodi applicativi, offrendo una soluzione flessibile e conforme agli standard di sicurezza di Aria S.p.A.

SpringLine 2 mette a disposizione dello sviluppatore un insieme di Authorities elencate nella tabella seguente:

Proprietà
"USERTYPE_" + userType
"CIRCUIT_" + circuit
"ROLE_" + role
"METHOD_" + method
"STRENGTH_" + strength
"CHANNEL_" + channel
"DATASOURCE_" + datasource
"CLAIM_" + key + "_" + claim (per ogni claim contenuto in claims)

N.B: userType, circuit, role, method, strength, channel, datasource, claims sono proprietà dell'oggetto User.

Per comprendere meglio come queste Authorities possano essere utilizzate, si propone un esempio di configurazione:

@Bean

```
SecurityFilterChain web(HttpSecurity http) throws Exception {  
    http  
        .authorizeHttpRequests((authorize) -> authorize  
            .requestMatchers("/endpoint").hasAuthority("CIRCUIT_R")  
            .anyRequest().authenticated()  
        );  
    return http.build();  
}
```

Il metodo **authorizeHttpRequests** è utilizzato per definire le autorizzazioni per le richieste HTTP. In questo specifico contesto, si richiede che le richieste rivolte a "/endpoint" siano autorizzate solo per gli utenti con l'autorità "CIRCUIT_R".

Questo approccio offre un controllo granulare sull'accesso alle risorse e ai metodi applicativi, consentendo agli sviluppatori di definire regole specifiche basate sui diritti associati a ciascun utente.

5.5.5 Configurazioni applicative YAML/Properties

Le configurazioni definiscono le opzioni per abilitare i diversi metodi di autenticazione. Inizialmente, tutti i metodi di autenticazione sono disabilitati per impostazione predefinita, il che significa che nessun sistema di autenticazione è considerato accettabile (enabled = false). Per abilitarli (enabled = true), è necessario sovrascrivere le impostazioni applicative e abilitare almeno un sistema di autenticazione. In caso contrario, qualsiasi API esposta restituirà un HttpStatus 403. È possibile abilitare (e quindi accettare) più sistemi di autenticazione contemporaneamente.

5.5.5.1 Anonymous

Proprietà	Default Value
spl.security.authentication.anonymous.enabled	true -> Anonymous accettato false -> Anonymous non accettato (default)
spl.security.authentication.anonymous.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è "/favicon.ico"

5.5.5.2 IAM

Proprietà	Default Value
spl.security.authentication.iam.enabled	true -> Anonymous accettato false -> Anonymous non accettato (default)
spl.security.authentication.iam.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è <code>"/**"</code>
spl.security.authentication.iam.header.codicefiscale	http_codicefiscale
spl.security.authentication.iam.header.matricola	http_matricola
spl.security.authentication.iam.header.nome	http_nome
spl.security.authentication.iam.header.cognome	http_cognome
spl.security.authentication.iam.header.email	http_email

5.5.5.3 API Manager

Proprietà	Default Value
spl.security.authentication.apimanager.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato (default)
spl.security.authentication.apimanager.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è <code>"/**"</code>
spl.security.authentication.apimanager.header.assertion	x-jwt-assertion
spl.security.authentication.apimanager.header.activcred	Activcred
spl.security.authentication.apimanager.header.uid	uid
spl.security.authentication.apimanager.header.token	token
spl.security.authentication.apimanager.header.tokenssodn	token_ssodn
spl.security.authentication.apimanager.header.ssourllogout	saml_attribute_ssourllogout

5.5.5.4 Propagator

Proprietà	Default Value
spl.security.authentication.propagator.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato (default)
spl.security.authentication.propagator.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è <code>"/**"</code>

5.5.5.5 IDPC

Proprietà	Default Value
-----------	---------------

Proprietà	Default Value
spl.security.authentication.idpc.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato
spl.security.authentication.idpc.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è <code>"/**"</code>
spl.security.authentication.idpc.header.codicefiscale	saml_attribute_codicefiscale
spl.security.authentication.idpc.header.nome	saml_attribute_nome
spl.security.authentication.idpc.header.cognome	saml_attribute_cognome
spl.security.authentication.idpc.header.cartareale	saml_attribute_cns_carta_reale
spl.security.authentication.idpc.header.tipoautenticazione	saml_attribute_tipoautenticazione
spl.security.authentication.idpc.header.livello-verifica	saml_attribute_livelloverifica
spl.security.authentication.idpc.header.email-address	saml_attribute_emailaddress
spl.security.authentication.idpc.header.ssourllogout	saml_attribute_ssourllogout
spl.security.authentication.idpc.header.nomeutente	saml_attribute_nomeutente
spl.security.authentication.idpc.header.sesso	saml_attribute_sesso
spl.security.authentication.idpc.header.datanascita	saml_attribute_datanascita
spl.security.authentication.idpc.header.luogonascita	saml_attribute_luogonascita
spl.security.authentication.idpc.header.provincianascita	saml_attribute_provincianascita
spl.security.authentication.idpc.header.statonascita	saml_attribute_statonascita
spl.security.authentication.idpc.header.cellulare	saml_attribute_cellulare
spl.security.authentication.idpc.header.ragionesociale	saml_attribute_ragionesociale
spl.security.authentication.idpc.header.sedelegale	saml_attribute_sedelegale
spl.security.authentication.idpc.header.partitaiva	saml_attribute_partitaiva
spl.security.authentication.idpc.header.docidentita	saml_attribute_docidentita
spl.security.authentication.idpc.header.scaddocidentita	saml_attribute_scaddocidentita
spl.security.authentication.idpc.header.domiciliofisico	saml_attribute_domiciliofisico
spl.security.authentication.idpc.header.domiciliodigitale	saml_attribute_domiciliodigitale
spl.security.authentication.idpc.header.cnsissuer	saml_attribute_cns_issuer
spl.security.authentication.idpc.header.cnssubject	saml_attribute_cns_subject

Proprietà	Default Value
spl.security.authentication.idpc.header.originedatiutente	saml_attribute_originedatiutente
spl.security.authentication.idpc.header.idcomuneregistrazione	saml_attribute_idcomuneregistrazione
spl.security.authentication.idpc.header.dataultimologin	data_ultimo_login
spl.security.authentication.idpc.header.oraultimologin	ora_ultimo_login
spl.security.authentication.idpc.header.indirizzoipcorrente	indirizzo_ip_corrente
spl.security.authentication.idpc.header.indirizzoipprecedente	indirizzo_ip_precedente
spl.security.authentication.idpc.header.tipospid	saml_attribute_tipo_spid
spl.security.authentication.idpc.header.servizioultimologin	saml_attribute_servizio_ultimo_login
spl.security.authentication.idpc.header.capdomiciliofisico	saml_attribute_capdomiciliofisico
spl.security.authentication.idpc.header.comunedomiciliofisico	saml_attribute_comunedomiciliofisico
spl.security.authentication.idpc.header.provinciadomiciliofisico	saml_attribute_provinciadomiciliofisico
spl.security.authentication.idpc.header.nazionedomiciliofisico	saml_attribute_nazionedomiciliofisico
spl.security.authentication.idpc.header.idpultimologin	saml_attribute_idp_ultimo_login
spl.security.authentication.idpc.header.organizzazione	organizzazione
spl.security.authentication.idpc.header.matricola	matricola
spl.security.authentication.idpc.header.enteappartenenza	enteappartenenza
spl.security.authentication.idpc.header.descrizioneente	descrizioneente
spl.security.authentication.idpc.header.statolavorativo	statolavorativo

5.5.5.6 SAMWEB

Proprietà	Default Value
spl.security.authentication.samweb.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato
spl.security.authentication.samweb.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è "/"
spl.security.authentication.samweb.header.userid	userid
spl.security.authentication.samweb.header.role	role
spl.security.authentication.samweb.header.activcred	activcred

Proprietà	Default Value
spl.security.authentication.samweb.header.uid	uid
spl.security.authentication.samweb.header.token	token
spl.security.authentication.samweb.header.tokenssodn	token_ssodn
spl.security.authentication.samweb.header.ssourlogout	saml_attribute_ssourlogout

5.5.5.7 JWT

Proprietà	Default Value
spl.security.authentication.jwt.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato (default)
spl.security.authentication.jwt.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è "/"
spl.security.authentication.jwt.header.authorization-bearer	Authorization
spl.security.authentication.jwt.header.enabled*	true -> viene accettato il jwt presente nell'header (default) false -> non viene rilevato il jwt presente nell'header
spl.security.authentication.jwt.cookie.enabled*	true -> viene accettato il jwt presente nel cookie false -> non viene rilevato il jwt presente nel cookie (default)
spl.security.authentication.jwt.cookie.name*	springline2Token

* dalle versioni 2024.02.01 e 2023.03.01

5.5.5.8 SpringLine

Proprietà	Default Value
spl.security.authentication.springline.enabled	true -> sistema di autenticazione accettato false -> Sistema di autenticazione non accettato (default)
spl.security.authentication.springline.uri-matchers	Indicare i paths da proteggere. Ogni path deve essere separato da virgola. Il valore di default è "/"
spl.security.authentication.springline.header.username	x-springline-username
spl.security.authentication.springline.header.authchannel	x-springline-authchannel
spl.security.authentication.springline.header.authmethod	x-springline-authmethod
spl.security.authentication.springline.header.authstrength	x-springline-authstrength
spl.security.authentication.springline.header.role	x-springline-role
spl.security.authentication.springline.header.firstname	x-springline-firstname

Proprietà	Default Value
spl.security.authentication.springline.header.lastname	x-springline-lastname
spl.security.authentication.springline.header.realvirtual	x-springline-realvirtual
spl.security.authentication.springline.header.fiscalcode	x-springline-fiscalcode
spl.security.authentication.springline.header.email	x-springline-email
spl.security.authentication.springline.header.credential	x-springline-credential
spl.security.authentication.springline.header.appname	x-springline-appname

5.5.6 Accesso alle informazioni dell'Utente

Con SpringLine 2, l'acquisizione e la gestione delle informazioni dell'utente autenticato sono state semplificate e automatizzate. Una volta completata l'autenticazione, SpringLine 2 si occupa di inizializzare e popolare il contesto per ogni transazione, archiviando le informazioni dell'utente sia nel contesto di SpringSecurity sia in quello dedicato di SpringLine 2. Nei paragrafi successivi, esploreremo le principali tecniche che gli sviluppatori possono adottare per accedere alle informazioni dell'utente, sia in modo completo che parziale.

5.5.6.1 Accesso mediante Contesto

Accedendo direttamente al contesto di SpringSecurity o SpringLine 2, gli sviluppatori hanno la possibilità di ottenere le informazioni dell'utente autenticato in qualsiasi punto dell'applicazione. Tuttavia, è fondamentale notare che, se una richiesta viene effettuata da una fonte esterna all'applicazione, come potrebbe essere un batch, il contesto non viene automaticamente generato. In tali scenari, gli sviluppatori devono manualmente inizializzare e chiudere il contesto per assicurare la corretta esecuzione dell'applicazione.

Di seguito un esempio di codice Java per accedere manualmente al contesto e ricavare l'oggetto User. Nell'esempio è definita una API Rest e lo scopo è quello di restituire la property name dell'utente loggato.

```
@GetMapping("/name")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.OK)
@Operation(description = "Restituisce il nome Utente",
    responses = { @ApiResponse(
        description = "Successful Operation",
        responseCode = "200",
        content = @Content(mediaType = "application/json")
    )
})
public ResponseEntity<String> getName() {
```

```

    User user = ContextHolder.currentContext().getUser();
    return RestResponse.ok(user.getName());
}

```

5.5.6.2 Accesso mediante Annotations

Per un accesso più granulare ai dati dell'utente, SpringLine 2 mette a disposizione le **annotations**. Consentono un accesso mirato e specifico ai dati dell'utente, eliminando la necessità di interagire con l'intero contesto. Tuttavia, esiste un'annotation che fornisce accesso completo all'entità dell'utente, permettendo di racchiudere tutte le properties in un singolo oggetto.

Quando si utilizzano le annotations all'interno dei Rest-Controller, SpringLine 2 gestisce automaticamente l'injection delle informazioni corrispondenti. E' compito dello sviluppatore assicurarsi che queste annotations non vengano percepite come input diretti del servizio, limitandone la visibilità.

5.5.6.2.1 Annotation User

Annotation
@AuthUser

5.5.6.2.2 Annotation Properties User

Annotation
@AuthAddress
@AuthAssertion
@AuthChannel
@AuthCircuit
@AuthCompanyName
@AuthCountryOfBirth
@AuthDataSource
@AuthDateOfBirth
@AuthDigitalAddress
@AuthDomicileMunicipality

Annotation	
@AuthDomicileNation	
@AuthDomicilePostalCode	
@AuthDomicileProvince	
@AuthDomicileStreetAddress	
@AuthEmail	
@AuthEmploymentStatus	
@AuthExpirationDate	
@AuthFamilyName	
@AuthFiscalNumber	
@AuthGender	
@AuthIdCard	
@AuthInstitutionDescription	
@AuthInstitutionName	
@AuthIpAddress	
@AuthIssuer	
@AuthIvaCode	
@AuthMethod	
@AuthMobilePhone	
@AuthName	
@AuthNationOfBirth	
@AuthOrganization	
@AuthPlaceOfBirth	
@AuthPreviousIdp	
@AuthPreviousIpAddress	
@AuthPreviousLoginDate	
@AuthPreviousLoginUrl	

Annotation
@AuthPreviousMethod
@AuthRegisteredOffice
@AuthRole
@AuthSafeCredential
@AuthSerialNumber
@AuthStrength
@AuthSubject
@AuthToken
@AuthTokenSsoDn
@AuthTrackId
@AuthUsername
@AuthUserType

Nell'esempio di codice Java riportato di seguito, sarà usata l'annotation @AuthName.

```

@GetMapping("/name")
@Transactional(readonly = true)
@ResponseStatus(HttpStatus.OK)
@Operation(description = "Restituisce il nome Utente",
    responses = { @ApiResponse(
        description = "Successful Operation",
        responseCode = "200",
        content = @Content(mediaType = "application/json"))
    })
public ResponseEntity<String> getName(@Parameter(hidden = true)
@AuthName String name) {
    return RestResponse.ok(name);
}

```

5.5.7 StrictHttpFirewall

Dalle versioni 2023.06.01 e 2024.05.01 di SpringLine 2 è presente un bean che crea un'istanza di StrictHttpFirewall, una classe che fornisce una protezione più rigorosa contro attacchi basati su HTTP. Questo bean personalizzato consente di definire un insieme personalizzato di valori consentiti per gli header HTTP. Convertire gli header da ISO-8859-1 a UTF-8 e verifica se corrispondono a una espressione regolare specificata.

Proprietà	Default Value
spl.security.http-firewall.enabled	true -> custom firewall attivo false -> custom firewall disattivato (default)
spl.security.http-firewall.regex	Espressione regolare per il match dei valori degli header. Il valore di default è [\\p{IsAssigned}&&[^[\\p{IsControl}]] \\t]

5.6 Hardening Tomcat Embedded

Springline 2 propone la seguente configurazione di hardening del tomcat embedded:

```
management.endpoints.web.exposure.include=health
management.endpoint.shutdown.enabled=false

server.servlet.context-path=/
server.error.whitelabel.enabled=true
server.error.include-exception=false
server.error.include-stacktrace=never

server.max-http-request-header-size=2MB
server.tomcat.max-http-form-post-size=2MB
server.tomcat.connection-timeout=5s
```

I valori in grassetto sono da adattare alle reali necessità applicative.

E' importante sottolineare che per avere una completa hardenizzazione è necessario che con venga modificato il context path, che deve essere "/".

6. SpringLine 2 - WS

Nell'ambito dell'integrazione con sistemi esterni, SpringLine 2 eleva il modulo 'spring-web-services' con una customizzazione mirata, che arricchisce il comportamento standard con funzionalità di logging e propagazione dell'utente connesso al sistema (solo per sistemi siss) secondo gli standard definiti da Aria S.p.A

Il fulcro di questa estensione è la customizzazione del WebServiceTemplate, che in SpringLine 2 viene dotato di queste due capacità aggiuntive. Oltre a svolgere le sue funzioni base di invio di richieste SOAP, il WebServiceTemplate personalizzato di SpringLine 2 genera (se configurato in modo opportuno) i log di monitoraggio/monitoraggio applicativo (MON e MON-APP) e propaga verso i servizi del mondo siss (servizi sviluppati con Handerground) i dati relativi all'utente connesso.

Di seguito sono riportate le configurazioni disponibili allo sviluppatore

6.1 Configurazioni

6.1.1 Logging MON

Proprietà	Valore
spl.client-ws.logging.mon.enabled	True/False Valore di default True
spl.client-ws.logging.mon.method-matchers	Valore di Default ** Dominio di valori: String Identifica tutti i remote method per le quali sarà tracciato il log. E' possibile definire l'elenco delle code separate da “,”
spl.client-ws.logging.mon.excluded-method-matchers	Valore di Default: Stringa Vuota Dominio di valori: String Identifica i remote method per i quali il log non sarà tracciato. E' possibile definire l'elenco dei method separati da “,”

6.1.2 Logging MON-APP

Proprietà	Valore
spl.client-ws.logging.app.enabled	True/False Valore di default False
spl.client-ws.logging.app.method-matchers	Valore di Default: ** Dominio di valori: String Identifica i remote method per i quali sarà tracciato il log. E' possibile definire l'elenco dei remote method separati da “,”
spl.client-ws.logging.app.excluded-method-matchers	Valore di Default: String Empty Dominio di valori: String Identifica i remote method per i quali il log non sarà tracciato. E' possibile definire l'elenco dei remote method separate da “,”
spl.client-ws.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per il quale il log sarà tracciato

6.1.3 Propagazione Siss

Proprietà	Valore
spl.client-ws.propagation.siss.method-matchers	Valore di Default: Stringa Vuota Dominio dei valori: String Identifica i remote method per i quali deve avvenire la propagazione. I valori, se molteplici devono essere separati da “,”
spl.client-ws.propagation.siss.excluded-method-matchers	Valore di Default: Stringa Vuota Dominio dei valori: String Identifica i remote method per i quali non deve avvenire la propagazione. I valori, se molteplici devono essere separati da “,”
spl.client-ws.propagation.siss.enabled	True/False Valore di Default: False
spl.client-ws.propagation.siss.header.userid	userid
spl.client-ws.propagation.siss.header.role	role
spl.client-ws.propagation.siss.header.activcred	activcred
spl.client-ws.propagation.siss.header.uid	uid
spl.client-ws.propagation.siss.header.token	token
spl.client-ws.propagation.siss.header.tokenssodn	token_ssodn
spl.client-ws.propagation.siss.header.ssourlogout	saml_attribute_ssourlogout

6.2 Esempi di codice - WS

6.2.1 Configurazione

Nell'esempio di seguito riportato sono state usate le variabili **contextPath** e **uriWs** valorizzate direttamente nella classe di Configurazione. Tale approccio è stato usato solo per questioni di semplicità. Si ricorda che secondo le linee guida Aria S.p.A, i valori devono essere esternalizzati.

@Configuration

```
public class CountryConfiguration {
    private String contextPath = "com.example.webservice";
    private String uriWs = "http://example.com/ws";
```

@Bean

```
/* default */ Jaxb2Marshaller marshaller() {
    Jaxb2Marshaller marshaller = new Jaxb2Marshaller();
    marshaller.setContextPath(contextPath);
    return marshaller;
}
```

```
@Bean
/* default */ CountryClient countryClient(Jaxb2Marshaller
                                         marshaller, WebServiceTemplate
                                         webServiceTemplate) {
    CountryClient client = new CountryClient();
    client.setWebServiceTemplate(webServiceTemplate);
    client.setDefaultUri(uriWs);
    client.setMarshaller(marshaller);
    client.setUnmarshaller(marshaller);
    return client;
}
}
```

6.2.2 Client SOAP

```
public class CountryClient extends WebServiceGatewaySupport {
    private String uri;
    private String soapAction;

    public CountryClient(String uri, String soapAction) {
        this.uri = uri;
        this.soapAction = soapAction;
    }

    public String getCurrency(final String country) {
        final GetCountryRequest request = new GetCountryRequest();
        request.setName(country);

        final GetCountryResponse response =
            (GetCountryResponse) getWebServiceTemplate()
                .marshalSendAndReceive(uri, request, new
                    SoapActionCallback(soapAction));
        return response.getCountry().getCurrency().value();
    }
}
```

7. SpringLine 2 - OpenApi

Il modulo OpenAPI di SpringLine 2 si afferma come un elemento fondamentale nell'architettura del software, essenziale per la documentazione delle API REST, in piena aderenza agli standard di ARIA S.p.A. Questo modulo si distingue per la sua capacità di integrarsi automaticamente con Swagger-UI, aggiungendo EndPoint all'applicazione che permettono non solo la visualizzazione ma anche la navigazione interattiva delle specifiche API. Inoltre, facilita la generazione di file JSON OpenAPI 3.0, fornendo una rappresentazione dettagliata delle informazioni specifiche del sistema software progettato e documentato dallo sviluppatore.

Una delle sue caratteristiche più rilevanti è la configurazione personalizzabile, che consente di attivare o disattivare il modulo, nonché di customizzare le informazioni esposte nello Swagger. Tutte le properties che SpringLine 2 offre allo sviluppatore per customizzare il modulo saranno riportate più avanti nel paragrafo nelle rispettive tabelle. La flessibilità si estende anche alla gestione dell'autenticazione in Swagger-UI, dove specifiche properties permettono di abilitare o disabilitare le funzionalità di sicurezza (per l'esecuzione delle API), evitando lo status Http 403 in assenza di configurazione.

È doveroso specificare che, per documentare le API, è essenziale che gli sviluppatori facciano riferimento alla documentazione ufficiale di Spring-Doc OpenAPI. Questa fornisce istruzioni dettagliate sull'uso delle annotazioni e definisce le properties per ulteriori personalizzazioni, garantendo che la documentazione delle API sia non solo completa ma anche in linea con le best practice.

In conclusione, questo modulo risponde alla necessità di una soluzione standardizzata e facilmente configurabile per la documentazione delle API REST, assicurando che tutti i progetti sviluppati sotto l'egida di ARIA S.p.A siano conformi alle migliori pratiche di documentazione delle API.

7.1.1 EndPoint

Di seguito saranno indicati gli endpoint disponibili dopo l'aggiunta dal modulo OpenApi.

URI
<Protocollo>://<Dominio>:<Porta>/swagger-ui/index.html
<Protocollo>://<Dominio>:<Porta>/v3/api-docs

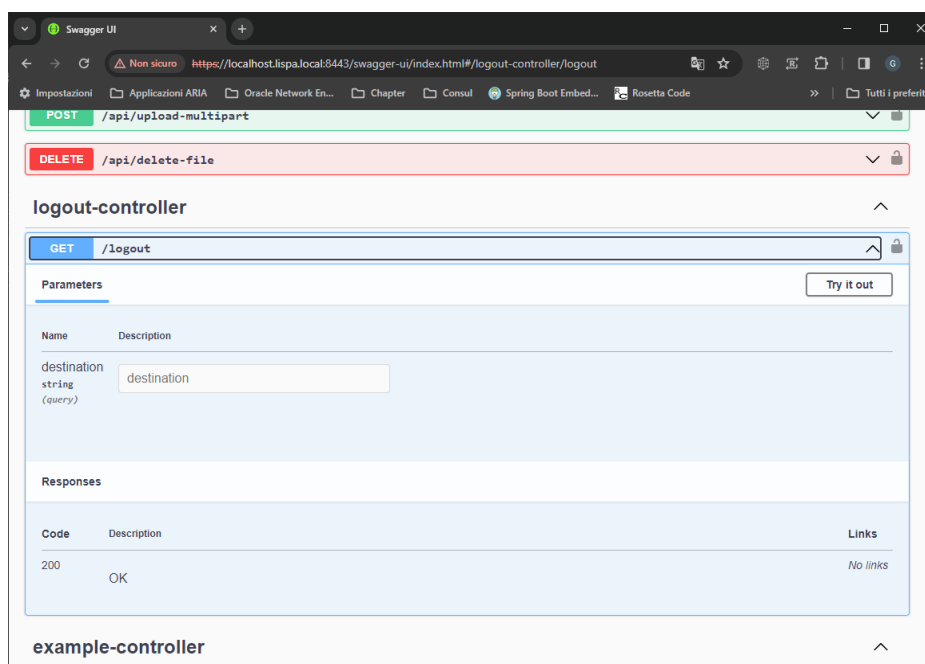
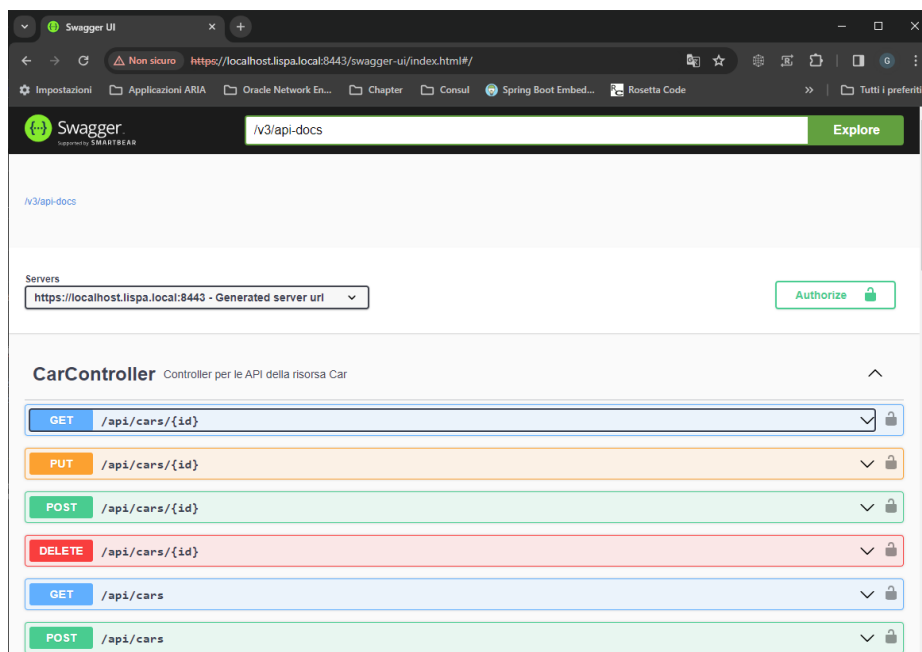
7.1.2 Configurazione

Proprietà	Valore
spl.open-api.configuration	True/False Il valore di Default è True. Consente di abilitare il modulo
spl.open-api.info.title	Default Stringa vuota
spl.open-api.info.description	Default Stringa vuota
spl.open-api.info.summary	Default Stringa vuota
spl.open-api.info.term-of-service	Default Stringa vuota
spl.open-api.info.term-of-service-url	Default Stringa vuota
spl.open-api.info.version	Default Stringa vuota
spl.open-api.info.contact.name	Default Stringa vuota
spl.open-api.info.contact.url	Default Stringa vuota
spl.open-api.info.contact.email	Default Stringa vuota
spl.open-api.info.license.name	Default Stringa vuota
spl.open-api.info.license.url	Default Stringa vuota
spl.open-api.info.license.identifier	Default Stringa vuota
spl.open-api.authorize.samweb	True/False Valore di Default False Consente di attivare il Sistema di Autenticazione SAMWEB in Swagger-UI e poter testare le rispettive API protette
spl.open-api.authorize.apimanager	True/False Valore di Default False Consente di attivare il Sistema di Autenticazione API MANAGER in Swagger-UI e poter testare le rispettive API protette
spl.open-api.authorize.iam	True/False Valore di Default False Consente di attivare il Sistema di Autenticazione IAM in Swagger-UI e poter testare le rispettive API protette

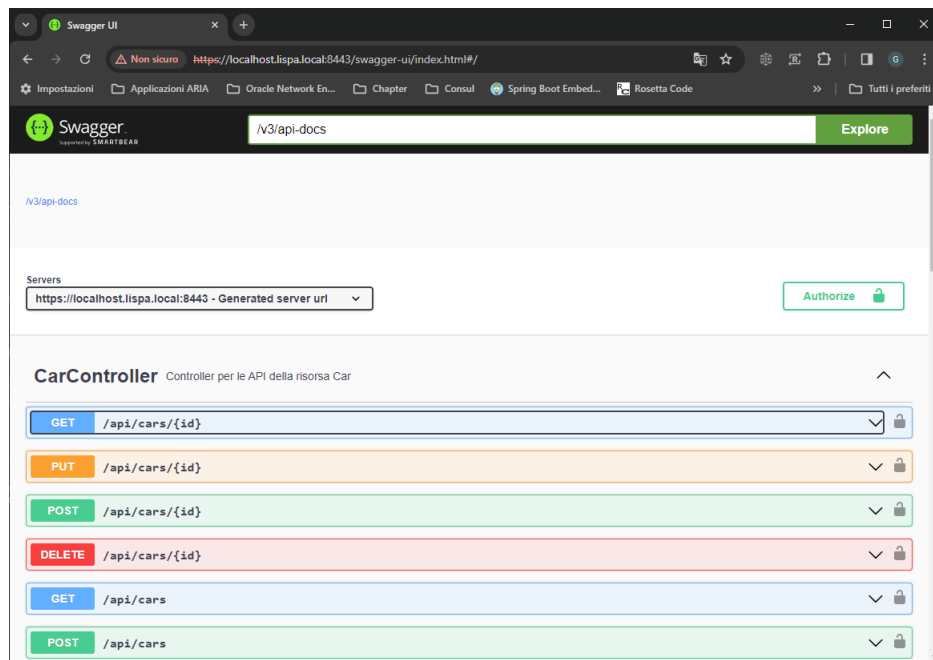
Proprietà	Valore
spl.open-api.authorize.idpc	True/False Valore di Default False Consente di attivare il Sistema di Autenticazione IDPC in Swagger-UI e poter testare le rispettive API protette

7.1.3 Esempi

Di seguito saranno mostrati degli esempi di interfaccia grafica di swagger-ui. Lo sviluppatore deve sostituire protocollo, dominio e porta rispetto all'ambiente dove swagger-ui è in esecuzione.



Nell'immagine di seguito riportata, è mostrato in swagger-ui il sistema di autenticazione abilitato (nel file di properties applicativo) che consente di eseguire le API.



8. SpringLine 2 – Data

SpringLine 2 - Data, un'estensione di Spring Boot Data JPA, arricchisce le capacità di gestione e monitoraggio dei dati in applicazioni Spring. Questo modulo, fondamentale per l'integrazione della connessione al database, amplifica le funzionalità di Spring Boot Data JPA, noto per la sua efficacia nell'astrazione dell'accesso ai dati e nella gestione delle transazioni, con caratteristiche aggiuntive:

- **Monitoraggio del Tempo di Esecuzione:** Aggiunge la capacità di monitorare e registrare nel log Mon il tempo impiegato dalle chiamate ai repository, dalla richiesta al risultato fornendo dati per l'analisi delle prestazioni. Tale informazione viene dapprima registrata nel contesto di SpringLine 2 nel campo "timeDB" e poi, se abilitato il log del MON, viene riportata nel campo TP_DB
- **Gestione di Database Reali e Virtuali:** Introduce la gestione di due tipi di datasource, uno reale e uno virtuale, particolarmente utile nel settore sanitario. Questa funzionalità consente di eseguire test in ambienti di produzione utilizzando dati virtuali, basandosi sull'identità dell'utente (virtuale o reale) gestita dagli Identity Providers: SAMWEB, APIMANAGER e IDPC.

Per la configurazione di SpringLine 2 - Data si definiscono le proprietà (a livello applicativo nel rispettivo file yaml) dei datasource reali e virtuali. Come tutti i moduli di SpringLine 2, è possibile attivare o disattivare il modulo Data mediante una specifica property (descritta nel seguito nell'apposita tabella di riferimento). In assenza di questa abilitazione, si procede con la configurazione standard del datasource di Spring. L'implementazione di SpringLine 2 - Data in un progetto Spring Boot rappresenta un importante avanzamento per gli sviluppatori, offrendo strumenti avanzati per un controllo più accurato e flessibile dell'accesso ai dati, in linea con le best practice e gli standard di Aria S.p.A.

8.1.1 Configurazione Data Source Yaml

spl:

datasource:

reale:

url:

username:

password:

hikari:

minimumIdle:

maximumPoolSize:

idleTimeout:

maxLifetime:

virtuale:

url:
username:
password:
hikari:
 minimumIdle:
 maximumPoolSize:
 idleTimeout:
 maxLifetime:

8.1.2 Configurazione Properties

Proprietà	Valore
spl.datasource.reale-virtuale	True/False Il valore di Default è False. Consente di abilitare la gestione della sorgente dati Reale/Virtuale
spl.datasource.logging.execution-time	True/False Il valore di Default è False. Consente di abilitare la gestione della sorgente dati Reale/Virtuale

9. SpringLine 2 – JMS

Il modulo SpringBoot ActiveMQ è una soluzione integrata essenziale per le applicazioni che necessitano di gestire operazioni asincrone basate su code e topic, facilitando l'interazione con ActiveMQ, un broker di messaggistica ampiamente utilizzato.

SpringLine 2 - JMS, estendendo le capacità di SpringBoot ActiveMQ, eleva l'integrazione con ActiveMQ a un nuovo livello. Questo modulo, in linea con le linee guida di Aria S.p.A., introduce una personalizzazione avanzata del client ActiveMQ, arricchendo l'applicativo con funzionalità di logging di monitoraggio (MON e MON-APP).

L'integrazione del modulo in un'applicazione offre agli sviluppatori:

1. la possibilità di incorporare un broker ActiveMQ direttamente nell'applicativo, ideale per scenari in cui l'applicazione:
 - a. è mono-istanza (in alternativa, è necessario un broker esterno).
 - b. funziona sia come produttore che come consumatore di messaggi.
2. usare le componenti client per accodare/scodare messaggi purché ci si connetta ad un Broker ActiveMQ esterno. In tal caso verrebbe meno la condizione della mono-istanza.

È importante considerare che:

1. la natura asincrona delle operazioni gestite da SpringLine 2 - JMS
2. l'assenza di meta tag che identifichino l'utente che ha richiesto un'operazione

il modulo tratta ogni transazione come eseguita da un utente anonimo.

Il modulo offre due componenti chiave:

1. SplJmsTemplate
2. Listener.

Dopo aver integrato l'applicativo con un broker ActiveMQ (sia embedded che esterno), il Listener rimane in ascolto e si occupa di scodare i messaggi da una determinata coda, mentre SplJmsTemplate è utilizzato per inviare messaggi a una coda o, sebbene meno consigliato, per leggerli, in quanto il Listener offre prestazioni superiori in termini di lettura.

Come per altri moduli in SpringLine 2, il modulo JMS offre una sequenza di properties configurabili che permettono agli sviluppatori di personalizzare ulteriormente il Listener secondo le specifiche esigenze dell'applicazione.

Per utilizzare SplJmsTemplate e Listener è necessario abilitarli impostando a true le proprietà

- `spl.jms.listener.enabled`
- `spl.jms.template.enabled`

Riferirsi al paragrafo 5.2.1) e 5.2.2) per consultare i specifici campi valorizzati nei rispettivi log di monitoraggio MON e MON-APP.

Dalle versioni 2026.01.01 e 2025.04.01 di SPRINGLINE2 è stato incluso il supporto per ActiveMQ Artemis in sostituzione di ActiveMQ Classic.

Di seguito è riportata la rispettiva tabella delle properties al fine di poter configurare il modulo.

9.1.1 Configurazione JMS Listener

Proprietà	Valore
spl.jms.listener.enabled	True/False Valore di default False
spl.jms.template.enabled	True/False Valore di default False
spl.jms.logging.mon.enabled	True/False Valore di default True
spl.jms.logging.mon.destination-matchers	Valore di default: ** Dominio di valori: String Identifica le code per le quali sarà tracciato il log E' possibile definire l'elenco degli end point separati da “,”
spl.jms.logging.mon.excluded-destination-matchers	Valore di Default: Stringa vuota Dominio di valori: String Identifica le code per i quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da “,”
spl.jms.logging.app.enabled	True/False Valore di default False
spl.jms.logging.app.destination-matchers	Valore di Default: ** Dominio di valori: String Identifica le code per i quali sarà tracciato il log E' possibile definire l'elenco delle code separate da “,”
spl.jms.logging.app.excluded-destination-matchers	Valore di Default: String Empty Dominio di valori: String Identifica le code per le quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da “,”
spl.jms.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

9.1.2 Configurazione JMS Client

Proprietà	Valore
-----------	--------

Proprietà	Valore
spl.client-jms.logging.mon.enabled	True/False Valore di default True
spl.client-jms.logging.mon.destination-matchers	Valore di Default ** Dominio di valori: String Identifica le code per le quali sarà tracciato il log E' possibile definire l'elenco delle code separate da ",",
spl.client-jms.logging.mon.excluded-destination-matchers	Valore di Default: Stringa vuota Dominio di valori: String Identifica le code per i quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da ",",
spl.client-jms.logging.app.enabled	True/False Valore di default False
spl.client-jms.logging.app.destination-matchers	Valore di Default: ** Dominio di valori: String Identifica le code per i quali sarà tracciato il log E' possibile definire l'elenco delle code separate da ",",
spl.client-jms.logging.app.excluded-destination-matchers	Valore di Default: String Empty Dominio di valori: String Identifica le code per le quali il log non sarà tracciato. E' possibile definire l'elenco delle code separate da ",",
spl.client-jms.logging.app.body-max-length	Default Value: 1048576 Dominio di valori: int Identifica la lunghezza massima del body per la quale il log sarà tracciato

9.1.3 Auditlog

Di seguito, sono riportate le property per i log di audit pre-valorizzate da SpringLine 2 a seguito di messaggi ricevuti dal listener jms:

Proprietà	Valore
AuditEvent	
AccessMode	"JMS"
AuditTransaction	
CodApp	spring.application.name
Method	Java method
IdInteraction	tracer.getTraceld() + "," + tracer.getSpanId());
SystemName	Nome Host
Rv	"R"
AuditUser	
SrcUser	"ANONYMOUS"

Proprietà	Valore
DstUser	"ANONYMOUS"
Username	"N/A"
UserRole	Se utente autentica Role
UserType	"ANONYMOUS"
AuthChannel	"NO_CHANNEL"
AuthMethod	"NO_METHOD"

10. SpringLine 2 – BATCH

Il modulo Spring Batch facilita lo sviluppo e l'esecuzione di processi batch nelle applicazioni Java. Un processo batch è un tipo di applicazione informatica in cui un insieme di dati è elaborato in un modo predefinito, senza interazione diretta con l'utente. Gli esempi comuni di processi batch includono l'elaborazione di grandi volumi di dati, la generazione di report periodici e l'aggiornamento di database.

Spring Batch semplifica la creazione di processi batch affidabili e scalabili fornendo una struttura di base e una serie di funzionalità comuni. Alcuni dei concetti chiave di Spring Batch includono:

Job: Un job è un insieme di passi che vengono eseguiti in sequenza per completare una determinata attività. Ogni passo può essere composto da letture di dati, trasformazioni e scritture di dati.

Step: Un passo è un'unità di lavoro all'interno di un job. Può comprendere operazioni come l'acquisizione di dati, la trasformazione e la scrittura su un target.

ItemReader: Un componente che legge i dati di input per il passo del job.

ItemProcessor: Un componente che esegue la trasformazione dei dati durante l'esecuzione del passo.

ItemWriter: Un componente che scrive i dati trasformati o processati su un sistema di destinazione, come un database o un file.

JobRepository: Un componente che gestisce lo stato e le informazioni sul lavoro, consentendo il recupero e la ripresa di job interrotti.

Spring Batch offre anche funzionalità di gestione delle transazioni, monitoraggio, registrazione, ripristino e altro ancora. Questo rende il framework particolarmente adatto per applicazioni che richiedono l'esecuzione periodica di attività batch, come l'elaborazione di dati di grandi dimensioni in background.

SpringLine 2 – BATCH, introdotto dalle versioni 2024.02.01 e 2023.03.01, rende disponibile un **Listener** che, in linea con le linee guida di Aria S.p.A., introduce una personalizzazione arricchendo i job di funzionalità di logging di audit e gestendo la creazione e distruzione dell'oggetto Context.

Nella configurazione dei job deve essere aggiunto tale listener (**ContextBatchListener**) in modo da consentire, allo start del job la scrittura del log di audit e l'inizializzazione del Context.

Di seguito è riportato un esempio:

```
}  
@Bean  
/* default */ Job processJob(JobRepository jobRepository,  
                             @Qualifier("step1") Step step1,  
                             @Qualifier("step2") Step step2,  
                             CustomJobExecutionDecider decider,  
                             ContextBatchListener listener) {  
    return new JobBuilder("carJob", jobRepository)  
        .start(step1)  
        .listener(listener)  
        .next(decider)  
        .on("STEP2")  
        .to(step2)  
        .from(decider)  
        .on("NO_STEP")  
        .end()  
        .build().build();  
}
```

11. Conclusione

Spring Line 2 rappresenta un passo avanti significativo rispetto alla sua versione precedente, offrendo una maggiore modularità e una vasta gamma di funzionalità aggiuntive. Questa flessibilità consente agli sviluppatori di costruire applicazioni robuste, rispettando gli standard qualitativi richiesti da Aria S.p.A.

12. Appendice A – Versioni

SPRINGLINE2 Version	SPRINGWIZ Version	Archetype	SpringBoot Version
2027.01.01	2027.01.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.5.5

SPRINGLINE2 Version	SPRINGWIZ Version	Archetype	SpringBoot Version
2026.02.01	2026.02.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.4.9
2026.01.01	2026.01.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.4.5

SPRINGLINE2 Version	SPRINGWIZ Version	Archetype	SpringBoot Version
2025.05.01	2025.05.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.13
2025.04.01	2025.04.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.11
2025.03.02	2025.03.01 ³	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.9

³ Dopo la generazione dell’archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

2025.03.01	2025.03.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.9
2025.02.01	2025.02.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.5
2025.01.02	2025.01.01 ⁴	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.3
2025.01.01	2025.01.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.3.3
SPRINGLINE2 Version	SPRINGWIZ Version	Archetype	SpringBoot Version
2024.08.02	2024.08.01 ⁵	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.12
2024.08.01	2024.08.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.12
2024.07.01	2024.07.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.11
2024.06.02	2024.06.01 ⁶	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.9
2024.06.01	2024.06.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.9
2024.05.03	2024.05.01 ⁷	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.6
2024.04.01	2024.04.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.4
2024.03.01	2024.03.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.3

⁴ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

⁵ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

⁶ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

⁷ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

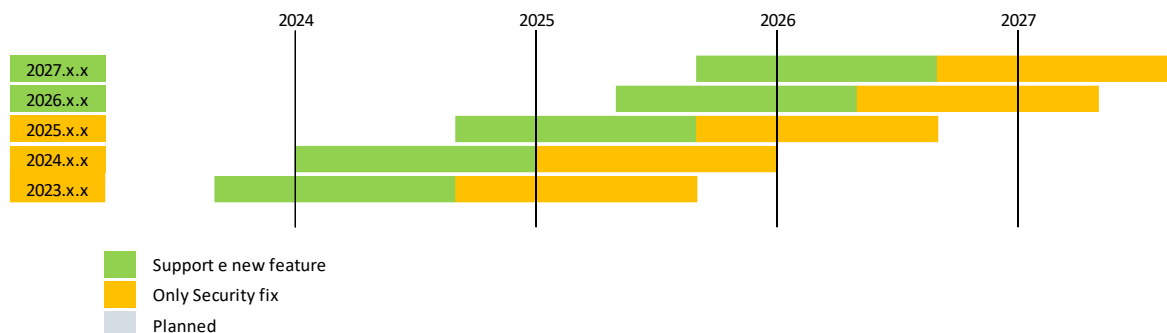
2024.02.01	2024.02.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.2.2
2024.01.01	2024.01.01	springwiz.api-archetype springwiz.www-archetype	3.2.1
SPRINGLINE2 Version	SPRINGWIZ Version	Archetype	SpringBoot Version
2023.06.05	2023.06.01 ⁸	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.12
2023.06.04	2023.06.01 ⁹	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.12
2023.06.03	2023.06.01 ¹⁰	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.12
2023.05.01	2023.05.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.10
2023.04.01	2023.04.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.9
2023.03.01	2023.03.01	springwiz.api-archetype springwiz.www-archetype springwiz.batch-archetype	3.1.8
2023.02.01	2023.02.01	springwiz.api-archetype springwiz.www-archetype	3.1.7
2023.01.01	2023.01.01	springwiz.api-archetype springwiz.www-archetype	3.1.6

Di seguito la policy del ciclo di vita di SPRINGLINE2:

⁸ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

⁹ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.

¹⁰ Dopo la generazione dell'archetype è necessario modificare il pom.xml per aggiornare la versione di SPRINGLINE2.



Branch	SpringBoot Version	Initial Release	End of Support	Only Security fix
● 2027.x.x	3.5.x	01/09/2025	01/09/2026	01/09/2027
● 2026.x.x	3.4.x	01/05/2025	01/05/2026	01/05/2027
● 2025.x.x	3.3.x	01/09/2024	01/09/2025	01/09/2026
● 2024.x.x	3.2.x	16/01/2024	31/12/2024	31/12/2025
● 2023.x.x	3.1.x	13/11/2023	30/08/2024	30/08/2025

13. Appendice B - Utilizzo degli archetype Maven

Per lo sviluppo delle applicazioni secondo gli ultimi standard architetturali, è possibile utilizzare gli archetype Maven che provvedono a generare in modo automatico un progetto di sviluppo corredato di tutte le librerie Spring Boot necessarie (comprese quelle della libreria SpringLine2).

Gli archetypes di riferimento sono i seguenti:

- **api**: utile alla creazione di una struttura di progetto per l'implementazione di servizi REST;
- **www**: utile alla creazione di una struttura di progetto che utilizza il framework Angular per l'implementazione di applicazioni Web che fanno uso di servizi REST;
- **batch**: utile alla creazione di una struttura di progetto per l'implementazione di procedure batch.

Per il loro utilizzo è sufficiente utilizzare il comando `mvn archetype:generate`: verrà generato un progetto da utilizzare come base di partenza per lo sviluppo delle applicazioni.

La sintassi completa del comando è la seguente:

```
mvn archetype:generate /
-DarchetypeGroupId=it.ariaspa.springwiz /
-DarchetypeArtifactId=<id archetype> /
```

```
-DarchetypeVersion=<versione archetype> /
-DcodiceApplicativoCodiceModuloElettra=<codice ELETTRA> /
-DinteractiveMode=false
```

Dove:

<id archetype> è l'archetype che si vuole utilizzare, e deve contenere uno dei seguenti valori:

- **springwiz.api-archetype**
- **springwiz.www-archetype**
- **springwiz.batch-archetype**

<versione archetype> è l'ultima versione disponibile.

<codice ELETTRA> è il codice dell'applicazione o del modulo così come catalogato in ELETTRA in formato minuscolo. Es **lsccs**, **nicce**, **prec**. È ammesso il codice del modulo dell'applicazione. Es. **sit.sidi**.

NOTA: nel caso si riscontrassero degli errori nella generazione dei progetti tramite l'archetype, si consiglia di svuotare il repository e ripetere l'operazione.

14. Appendice C – Default Application Properties

È possibile specificare varie proprietà all'interno del file `application.properties` o all'interno del file `application.yml` o come opzioni della riga di comando. Questa appendice fornisce l'elenco delle proprietà comuni di Spring Line 2 con il relativo valore di default.

```
spring.cloud.config.enabled=false
spring.cloud.bootstrap.enabled=true

logging.config=classpath:logback-springline2.xml
logging.file.path=logs
logging.level.root=INFO
logging.level.org.springframework.core.LocalVariableTableParameterNameDiscoverer=ERROR

spring.cloud.consul.enabled=false
spring.cloud.consul.discovery.enabled=false
spring.cloud.consul.service-registry.enabled=false
spring.cloud.consul.config.enabled=true
spring.cloud.consul.config.prefix=config-server
spring.cloud.consul.config.data-key=application.yml
spring.cloud.consul.config.format=YAML
```

```
spl.logging.encrypt.keyStore.location=
spl.logging.encrypt.keyStore.password=
spl.logging.encrypt.keyStore.type=
spl.logging.encrypt.keyStore.alias=

spl.conjur.enabled=true
spl.conjur.appliance-url=
spl.conjur.account=
spl.conjur.authn.login=
spl.conjur.authn.api-key=
spl.conjur.authn.url=
spl.conjur.authn.token-file=
spl.conjur.tls-ca-path=
spl.conjur.cache=false

spl.consul.cache.path=
spl.consul.cache.enabled=true

management.endpoints.web.exposure.include=health
management.endpoint.shutdown.access=none

spring.mvc.servlet.path=/
server.servlet.context-path=/

server.error.whitelabel.enabled=true
server.error.include-exception=false
server.error.include-stacktrace=never

server.max-http-request-header-size=2MB
server.tomcat.max-http-form-post-size=2MB
server.tomcat.connection-timeout=5s

spring.jpa.open-in-view=true

spring.artemis.embedded.enabled=false
spring.artemis.mode=NATIVE

spl.error.exception-handler.enabled=true
spl.error.exception-handler.auditlog=true

spl.security.generated-dev-password.enabled=false

spl.security.http-firewall.enabled=false
spl.security.http-firewall.regex=[\\p{IsAssigned}&&[\\p{IsControl}]]|\\t]*

spl.security.authentication.anonymous.enabled=false
spl.security.authentication.anonymous.uri-matchers=/favicon.ico

spl.security.authentication.propagator.enabled=false
spl.security.authentication.propagator.uri-matchers=/**

spl.security.authentication.springline.enabled=false
spl.security.authentication.springline.uri-matchers=/**
spl.security.authentication.springline.header.username=x-springline-username
spl.security.authentication.springline.header.authchannel=x-springline-authchannel
spl.security.authentication.springline.header.authmethod=x-springline-authmethod
spl.security.authentication.springline.header.authstrength=x-springline-authstrength
spl.security.authentication.springline.header.role=x-springline-role
spl.security.authentication.springline.header.firstname=x-springline-firstname
spl.security.authentication.springline.header.lastname=x-springline-lastname
```

```
spl.security.authentication.springline.header.realvirtual=x-springline-realvirtual
spl.security.authentication.springline.header.fiscalcode=x-springline-fiscalcode
spl.security.authentication.springline.header.email=x-springline-email
spl.security.authentication.springline.header.credential=x-springline-credential
spl.security.authentication.springline.header.appname=x-springline-appname

spl.security.authentication.samweb.enabled=false
spl.security.authentication.samweb.uri-matchers=/**
spl.security.authentication.samweb.header.activcred=activcred
spl.security.authentication.samweb.header.uid=uid
spl.security.authentication.samweb.header.token=token
spl.security.authentication.samweb.header.tokenssodn=token_ssodn
spl.security.authentication.samweb.header.ssourllogout=saml_attribute_ssourllogout

spl.security.authentication.idpc.enabled=false
spl.security.authentication.idpc.uri-matchers=/**
spl.security.authentication.idpc.header.codicefiscale=saml_attribute_codicefiscale
spl.security.authentication.idpc.header.nome=saml_attribute_nome
spl.security.authentication.idpc.header.cognome=saml_attribute_cognome
spl.security.authentication.idpc.header.cartareale=saml_attribute_cns_carta_reale
spl.security.authentication.idpc.header.tipoautenticazione=saml_attribute_tipoautenticazione
spl.security.authentication.idpc.header.livello-verifica=saml_attribute_livelloverifica
spl.security.authentication.idpc.header.email-address=saml_attribute_emailaddress
spl.security.authentication.idpc.header.ssourllogout=saml_attribute_ssourllogout
spl.security.authentication.idpc.header.nomeutente=saml_attribute_nomeutente
spl.security.authentication.idpc.header.sesso=saml_attribute_sesso
spl.security.authentication.idpc.header.datanascita=saml_attribute_datanascita
spl.security.authentication.idpc.header.luogonascita=saml_attribute_luogonascita
spl.security.authentication.idpc.header.provincianascita=saml_attribute_provincianascita
spl.security.authentication.idpc.header.statonascita=saml_attribute_statonascita
spl.security.authentication.idpc.header.cellulare=saml_attribute_cellulare
spl.security.authentication.idpc.header.ragionesociale=saml_attribute_ragionesociale
spl.security.authentication.idpc.header.sedelegale=saml_attribute_sedelegale
spl.security.authentication.idpc.header.partitaiva=saml_attribute_partitaiva
spl.security.authentication.idpc.header.docidentita=saml_attribute_docidentita
spl.security.authentication.idpc.header.scaddocidentita=saml_attribute_scaddocidentita
spl.security.authentication.idpc.header.domiciliofisico=saml_attribute_domiciliofisico
spl.security.authentication.idpc.header.domiciliodigitale=saml_attribute_domiciliodigitale
spl.security.authentication.idpc.header.cnsissuer=saml_attribute_cns_issuer
spl.security.authentication.idpc.header.cnssubject=saml_attribute_cns_subject
spl.security.authentication.idpc.header.originedatiutente=saml_attribute_originedatiutente
spl.security.authentication.idpc.header.idcomuneregistrazione=saml_attribute_idcomuneregistrazione
spl.security.authentication.idpc.header.dataultimologin=data_ultimo_login
spl.security.authentication.idpc.header.oraultimologin=ora_ultimo_login
spl.security.authentication.idpc.header.indirizzoipcorrente=indirizzo_ip_corrente
spl.security.authentication.idpc.header.indirizzoipprecedente=indirizzo_ip_precedente
spl.security.authentication.idpc.header.tipospid=saml_attribute_tipo_spid
spl.security.authentication.idpc.header.servizioultimologin=saml_attribute_servizio_ultimo_login
spl.security.authentication.idpc.header.capdomiciliofisico=saml_attribute_capdomiciliofisico
spl.security.authentication.idpc.header.comunedomiciliofisico=saml_attribute_comunedomiciliofisico
spl.security.authentication.idpc.header.provinciadomiciliofisico=saml_attribute_provinciadomiciliofisico
spl.security.authentication.idpc.header.nazionedomiciliofisico=saml_attribute_nazionedomiciliofisico
spl.security.authentication.idpc.header.idpultimologin=saml_attribute_idp_ultimo_login
spl.security.authentication.idpc.header.organizzazione=organizzazione
spl.security.authentication.idpc.header.matricola=matricola
spl.security.authentication.idpc.header.enteappartenenza=enteappartenenza
spl.security.authentication.idpc.header.descrizioneente=descrizioneente
spl.security.authentication.idpc.header.statolavorativo=statolavorativo

spl.security.authentication.iam.enabled=false
spl.security.authentication.iam.uri-matchers=/**
```

```
spl.security.authentication.iam.header.codicefiscale=http_codicefiscale
spl.security.authentication.iam.header.matricola=http_matricola
spl.security.authentication.iam.header.nome=http_nome
spl.security.authentication.iam.header.cognome=http_cognome
spl.security.authentication.iam.header.email=http_email

spl.security.authentication.apimanager.enabled=false
spl.security.authentication.apimanager.uri-matchers=/**
spl.security.authentication.apimanager.header.assertion=x-jwt-assertion
spl.security.authentication.apimanager.header.uid=uid
spl.security.authentication.apimanager.header.token=token
spl.security.authentication.apimanager.header.tokenssodn=token_ssodn
spl.security.authentication.apimanager.header.ssourllogout=saml_attribute_ssourllogout
spl.security.authentication.apimanager.header.activcred=activcred

spl.security.authentication.jwt.enabled=false
spl.security.authentication.jwt.uri-matchers=/**
spl.security.authentication.jwt.header.authorization-bearer=authorization

spl.security.authentication.jwt.header.enabled=true
spl.security.authentication.jwt.cookie.enabled=false
spl.security.authentication.jwt.cookie.name=springline2Token

spl.security.jwt.cypher-secret=
spl.security.jwt.token-validity=1800

spl.datasource.logging.execution-time=false
spl.datasource.reale-virtuale=false

spl.open-api.configuration=true
spl.open-api.info.title=
spl.open-api.info.description=
spl.open-api.info.summary=
spl.open-api.info.term-of-service=
spl.open-api.info.term-of-service-url=
spl.open-api.info.version=
spl.open-api.info.contact.name=
spl.open-api.info.contact.url=
spl.open-api.info.contact.email=
spl.open-api.info.license.name=
spl.open-api.info.license.url=
spl.open-api.info.license.identifier=
spl.open-api.authorize.samweb=false
spl.open-api.authorize.apimanager=false
spl.open-api.authorize.iam=false
spl.open-api.authorize.idpc=false

spl.http.logging.mon.enabled=true
spl.http.logging.mon.uri-matchers=/**
spl.http.logging.mon.excluded-uri-matchers=/favicon.ico
spl.http.logging.app.enabled=false
spl.http.logging.app.uri-matchers=/**
spl.http.logging.app.excluded-uri-matchers=/favicon.ico
spl.http.logging.app.excluded-content-types=application/x-www-form-urlencoded,multipart/form-data
spl.http.logging.app.body-max-length=1048576
spl.http.logging.app.add-request-uri=false

spl.jms.listener.enabled=false
spl.jms.template.enabled=false
spl.jms.logging.mon.enabled=true
spl.jms.logging.mon.destination-matchers=**
```

```
spl.jms.logging.mon.excluded-destination-matchers=
spl.jms.logging.app.enabled=false
spl.jms.logging.app.destination-matchers=**
spl.jms.logging.app.excluded-destination-matchers=
spl.jms.logging.app.body-max-length=1048576

spl.client-rest.autoconfiguration.enabled=true

spl.client-rest.logging.mon.enabled=true
spl.client-rest.logging.mon.uri-matchers=/**
spl.client-rest.logging.mon.excluded-uri-matchers=
spl.client-rest.logging.app.enabled=false
spl.client-rest.logging.app.uri-matchers=/**
spl.client-rest.logging.app.excluded-uri-matchers=
spl.client-rest.logging.app.excluded-content-types=
spl.client-rest.logging.app.body-max-length=1048576

spl.client-rest.propagation.springline2.enabled=false
spl.client-rest.propagation.springline2.uri-matchers=
spl.client-rest.propagation.springline2.excluded-uri-matchers=

spl.client-rest.propagation.springline.enabled=false
spl.client-rest.propagation.springline.uri-matchers=
spl.client-rest.propagation.springline.excluded-uri-matchers=
spl.client-rest.propagation.springline.header.username=x-springline-username
spl.client-rest.propagation.springline.header.authchannel=x-springline-authchannel
spl.client-rest.propagation.springline.header.authmethod=x-springline-authmethod
spl.client-rest.propagation.springline.header.authstrength=x-springline-authstrength
spl.client-rest.propagation.springline.header.role=x-springline-role
spl.client-rest.propagation.springline.header.firstname=x-springline-firstname
spl.client-rest.propagation.springline.header.lastname=x-springline-lastname
spl.client-rest.propagation.springline.header.realvirtual=x-springline-realvirtual
spl.client-rest.propagation.springline.header.fiscalcode=x-springline-fiscalcode
spl.client-rest.propagation.springline.header.email=x-springline-email
spl.client-rest.propagation.springline.header.credential=x-springline-credential
spl.client-rest.propagation.springline.header.appname=x-springline-appname

spl.client-rest.retry.uri-matchers=/**
spl.client-rest.retry.excluded-uri-matchers=
spl.client-rest.retry.enabled=false
spl.client-rest.retry.max-retries=3
spl.client-rest.retry.with-timeout=0
spl.client-rest.retry.fixed-backoff=0

spl.client-jms.logging.mon.enabled=true
spl.client-jms.logging.mon.destination-matchers=**
spl.client-jms.logging.mon.excluded-destination-matchers=
spl.client-jms.logging.app.enabled=false
spl.client-jms.logging.app.destination-matchers=**
spl.client-jms.logging.app.excluded-destination-matchers=
spl.client-jms.logging.app.body-max-length=1048576

spl.client-ws.autoconfiguration.enabled:true

spl.client-ws.propagation.siss.enabled=false
spl.client-ws.propagation.siss.method-matchers=
spl.client-ws.propagation.siss.excluded-method-matchers=
spl.client-ws.propagation.siss.header.userid=userid
spl.client-ws.propagation.siss.header.role=role
spl.client-ws.propagation.siss.header.activcred=activcred
spl.client-ws.propagation.siss.header.uid=uid
```

```
spl.client-ws.propagation.siss.header.token=token
spl.client-ws.propagation.siss.header.tokenssodn=token_ssodn
spl.client-ws.propagation.siss.header.ssourllogout=saml_attribute_ssourllogout

spl.client-ws.logging.mon.enabled=true
spl.client-ws.logging.mon.method-matchers=**
spl.client-ws.logging.mon.excluded-method-matchers=
spl.client-ws.logging.app.enabled=false
spl.client-ws.logging.app.method-matchers=**
spl.client-ws.logging.app.excluded-method-matchers=
spl.client-ws.logging.app.body-max-length=1048576
```

15. Appendice D – CHANGELOG

**** 2027.01.01**

- * ALL spring boot 3.5.5
- * CORE spring cloud 2025.0.0
- * CORE Possibilità di aggiungere nel campo IN del log APP la requestURI completa di query string nel formato method#requestURI#queryString#body
 - Nuova proprietà spl.http.logging.app.add-request-uri con valore di default false per abilitare questa funzione
- * AUDITLOG jackson 2.19.2
- * OPENAPI springdoc-openapi-starter 2.8.13
- * OPENAPI Aggiunta registrazione modulo Jdk8Module in Jackson per gestione Optional
- * CONFIG conjur api 3.1.0
- * CONFIG spring cloud 2025.0.0
- * JMS il valore di default della property spl.jms.listener.enabled è stato impostato a false
- * JMS nuova property spl.jms.template.enabled con valore default a false per abilitare il bean SPLJmsTemplate in alternativa a JMSTemplate. Questo bean implementa i log MON e APP nelle comunicazioni con code e topic.

**** 2026.02.01**

- * ALL spring boot 3.4.9
- * CORE spring cloud 2024.0.2
- * CORE Possibilità di aggiungere nel campo IN del log APP la requestURI completa di query string nel formato method#requestURI#queryString#body
 - Nuova proprietà spl.http.logging.app.add-request-uri con valore di default false per abilitare questa funzione
- * CONFIG spring cloud 2024.0.2
- * CONFIG conjur api 3.1.0
- * OPENAPI springdoc-openapi-starter 2.8.9
- * OPENAPI Aggiunta registrazione modulo Jdk8Module in Jackson per gestione Optional
- * JMS il valore di default della property spl.jms.listener.enabled è stato impostato a false
- * JMS nuova property spl.jms.template.enabled con valore default a false per abilitare il bean SPLJmsTemplate in alternativa a JMSTemplate. Questo bean implementa i log MON e APP nelle comunicazioni con code e topic.
- * AUDITLOG jackson 2.18.4

**** 2026.01.01**

- * ALL spring boot 3.4.5
- * CORE spring cloud 2024.0.1
- * CORE possibilità di associare e gestire dati applicativi arbitrari all'interno del contesto corrente (Context), offrendo maggiore flessibilità nello scambio di informazioni tra le componenti dell'applicazione
 - È stata aggiunta una mappa interna (private final Map<String, Object> attributes) alla classe Context. Questa mappa serve a contenere dati applicativi generici, identificati da una chiave di tipo String.
 - È stata introdotta una nuova annotation @CtxAttributes. Questa annotation è pensata per essere utilizzata sui parametri dei metodipermette di iniettare direttamente la mappa attributes del contesto corrente.
- * OPENAPI springdoc-openapi-starter 2.8.8
- * AUDITLOG jackson 2.18.3
- * AUDITLOG logback 1.5.18
- * AUDITLOG sfl4j 2.0.17
- * CONFIG spring cloud 2024.0.1
- * CONFIG conjur api 3.0.6
- * JMS Sostituito ActiveMQ Classic con ActiveMQ Artemis

**** 2025.05.01**

- * ALL spring boot 3.3.13
- * CORE Possibilità di aggiungere nel campo IN del log APP la requestURI completa di query string nel formato method#requestURI#queryString#body
 - Nuova proprietà spl.http.logging.app.add-request-uri con valore di default false per abilitare questa funzione
- * CORE spring cloud 2023.0.5
- * CONFIG conjur api 3.1.0
- * CONFIG spring cloud 2023.0.5
- * JMS il valore di default della property spl.jms.listener.enabled è stato impostato a false
- * JMS nuova property spl.jms.template.enabled con valore default a false per abilitare il bean SPLJmsTemplate in alternativa a JMSTemplate. Questo bean implementa i log MON e APP nelle comunicazioni con code e topic.

**** 2025.04.01**

- * ALL spring boot 3.3.11
- * CORE possibilità di associare e gestire dati applicativi arbitrari all'interno del contesto corrente (Context), offrendo maggiore flessibilità nello scambio di informazioni tra le componenti dell'applicazione
 - È stata aggiunta una mappa interna (private final Map<String, Object> attributes) alla classe Context. Questa mappa serve a contenere dati applicativi generici, identificati da una chiave di tipo String.
 - È stata introdotta una nuova annotation @CtxAttributes. Questa annotation è pensata per essere utilizzata sui parametri dei metodipermette di iniettare direttamente la mappa attributes del contesto corrente.
- * AUDITLOG logback 1.5.18

- * AUDITLOG sfl4j 2.0.17
- * CONFIG conjur api 3.0.6
- * JMS Sostituito ActiveMQ Classic con ActiveMQ Artemis

** 2025.03.02

- * CORE Corretto bug che causava un troncamento della response in caso di esclusione di mimetype

** 2025.03.01

- * ALL spring boot 3.3.9
- * CORE spring cloud 2023.0.5
- * CORE Con autenticazione API Manager, applicationName e traceId dell'oggetto Tracer sono valorizzati dal claim applicationname e dall'header UID.
- * CORE Con autenticazione SAMWEB, traceId dell'oggetto Tracer è valorizzato dall'header UID.
- * AUDITLOG jackson 2.17.3
- * AUDITLOG logback 1.5.12
- * CONFIG spring cloud 2023.0.5
- * CONFIG Gestione della cache locale cifrata per le properties lette da Hashicorp Consul
 - Nuova proprietà spl.consul.cache.enabled con valore default true
 - Nuova proprietà spl.consul.cache.path

** 2025.02.01

- * ALL spring boot 3.3.5
- * CORE Autenticazione API Manager
 - Nuova proprietà spl.security.authentication.apimanager.header.uid con valore default uid
 - Nuova proprietà spl.security.authentication.apimanager.header.token con valore default token
 - Nuova proprietà spl.security.authentication.apimanager.header.tokenssodn con valore default tokenssodn
 - Nuova proprietà spl.security.authentication.apimanager.header.ssourllogout con valore default

saml_attribute_ssourllogout

- * CORE Nuovo logback converter per cifrare il contenuto del messaggio di log
 - Nuova classe it.ariaspa.springline2.logging.logback.EncryptMsgConverter
 - Nuovo file di configurazione logback-springline2-encrypt.xml
 - Nuovo file di configurazione logback-springline2-ocp-encrypt.xml
 - Nuova proprietà spl.logging.encrypt.keyStore.location
 - Nuova proprietà spl.logging.encrypt.keyStore.password
 - Nuova proprietà spl.logging.encrypt.keyStore.type
 - Nuova proprietà spl.logging.encrypt.keyStore.alias
- * CORE Eliminata gestione cache locale delle properties recuperate da Spring Cloud Config Server
 - Eliminata proprietà spl.cloud.config.local-cache.enabled
- * AUDITLOG logback 1.5.11
- * CONFIG nuovo modulo per la gestione delle properties in Cyberark Conjur e in Hashicorp Consul
 - Nuova proprietà spl.conjur.enabled con valore default true
 - Nuova proprietà spl.conjur.appliance-url
 - Nuova proprietà spl.conjur.account
 - Nuova proprietà spl.conjur.authn.login
 - Nuova proprietà spl.conjur.authn.api-key
 - Nuova proprietà spl.conjur.authn.url
 - Nuova proprietà spl.conjur.authn.token-file
 - Nuova proprietà spl.conjur.tls-ca-path
 - Nuova proprietà spl.conjur.cache con valore default false

** 2025.01.02

- * CORE Corretto bug che, in caso di autenticazione API MANAGER con chiavi SANDBOX, impostava il circuito REALE invece che VIRTUALE
- * CORE Modificata la sequenza dei filter che gestiscono l'autenticazione per gestire il caso in cui un endpoint è esposto sia in IPDC che SSO

** 2025.01.01

- * ALL spring boot 3.3.3
- * AUDITLOG jackson 2.17.2
- * AUDITLOG logback 1.5.7

** 2024.08.02

- * CORE Corretto bug che causava un troncamento della response in caso di esclusione di mimetype

** 2024.08.01

- * ALL spring boot 3.2.12
- * CORE spring cloud 2023.0.5
- * CORE Con autenticazione API Manager, applicationName e traceId dell'oggetto Tracer sono valorizzati dal claim applicationname e dall'header UID.
- * CORE Con autenticazione SAMWEB, traceId dell'oggetto Tracer è valorizzato dall'header UID.
- * CONFIG spring cloud 2023.0.5
- * CONFIG Gestione della cache locale cifrata per le properties lette da Hashicorp Consul
 - Nuova proprietà spl.consul.cache.enabled con valore default true
 - Nuova proprietà spl.consul.cache.path

** 2024.07.01

- * ALL spring boot 3.2.11
- * CORE Autenticazione API Manager
 - Nuova proprietà spl.security.authentication.apimanager.header.uid con valore default uid
 - Nuova proprietà spl.security.authentication.apimanager.header.token con valore default token
 - Nuova proprietà spl.security.authentication.apimanager.header.tokenssodn con valore default tokenssodn
 - Nuova proprietà spl.security.authentication.apimanager.header.ssourllogout con valore default

saml_attribute_ssourllogout

- * CORE Nuovo logback converter per cifrare il contenuto del messaggio di log
 - Nuova classe it.ariaspa.springline2.logging.logback.EncryptMsgConverter
 - Nuovo file di configurazione logback-springline2-encrypt.xml
 - Nuovo file di configurazione logback-springline2-ocp-encrypt.xml
 - Nuova proprietà spl.logging.encrypt.keyStore.location
 - Nuova proprietà spl.logging.encrypt.keyStore.password
 - Nuova proprietà spl.logging.encrypt.keyStore.type
 - Nuova proprietà spl.logging.encrypt.keyStore.alias
- * CORE Eliminata gestione cache locale delle properties recuperate da Spring Cloud Config Server
 - Eliminata proprietà spl.cloud.config.local-cache.enabled
- * CONFIG nuovo modulo per la gestione delle properties in Cyberark Conjur e in Hashicorp Consul
 - Nuova proprietà spl.conjur.enabled con valore default true
 - Nuova proprietà spl.conjur.appliance-url
 - Nuova proprietà spl.conjur.account
 - Nuova proprietà spl.conjur.authn.login
 - Nuova proprietà spl.conjur.authn.api-key
 - Nuova proprietà spl.conjur.authn.url
 - Nuova proprietà spl.conjur.authn.token-file
 - Nuova proprietà spl.conjur.tls-ca-path
 - Nuova proprietà spl.conjur.cache con valore default false

** 2024.06.02

- * CORE Corretto bug che, in caso di autenticazione API MANAGER con chiavi SANDBOX, impostava il circuito REALE invece che VIRTUALE

* CORE Modificata la sequenza dei filter che gestiscono l'autenticazione per gestire il caso in cui un endpoint è esposto sia in IPDC che SSO

** 2024.06.03

- * CORE Corretto bug che, in caso di autenticazione API MANAGER con chiavi SANDBOX, impostava il circuito REALE invece che VIRTUALE
- * CORE Modificata la sequenza dei filter che gestiscono l'autenticazione per gestire il caso in cui un endpoint è esposto sia in IPDC che SSO

** 2024.06.01

- * ALL spring boot 3.2.9
- * CORE spring cloud 2023.0.3
- * CORE jjwt 0.12.6
- * OPENAPI springdoc-openapi-starter 2.6.0
- * AUDITLOG logback 1.4.14
- * AUDITLOG slf4j 2.0.16

** 2024.05.03

- * CORE Gestione exception MissingServletRequestPartException per ritorno status code 400.
- * CORE modificato valore di default della proprietà spl.http.logging.app.excluded-content-types:application/x-www-form-urlencoded,multipart/form-data
- * CORE Corretto bug che causava NullPointerException durante l'uso del RestTemplate in caso di content type null
- * CORE Modificata la gestione della esclusione del log APP in funzione del content type.

** 2024.05.02

- * CORE Gli header HTTP senza valore sono ora considerati assenti
- * CORE Corretto bug che causava NullPointerException in caso di credenziale operatore senza L1 o L2 o L3 (autenticazione API Manager)
- * AUDITLOG jackson 2.15.4
- * AUDITLOG slf4j 2.0.13

** 2024.05.01

- * ALL spring boot 3.2.6
- * CORE spring cloud 2023.0.1
- * CORE Aggiunto un custom StrictHttpFirewall che converte in UTF-8 i valori degli header http
 - Nuova proprietà spl.security.http-firewall.enabled con valore default false
 - Nuova proprietà spl.security.http-firewall.regex con valore default `[\\p{IsAssigned}&{[\\^\\p{IsControl}]]|\\t]*`
- * CORE Se viene impostato a DEBUG il log della classe `it.ariaspa.springline2.filter.TracerFilter` vengono loggate informazioni della request
- * CORE Aggiunta variabile (traceId) MDC e valorizzata con i valori di trace
- * CORE Modificato pattern layout del log in console e file per stampare la variabile MDC traceId
- * OPENAPI springdoc-openapi-starter 2.4.0

** 2024.04.01

- * ALL spring boot 3.2.4
- * CORE jjwt 0.12.5
- * CORE Migliorata la risposta in caso di errore di validazione dell'input
- * CORE Corretto bug che causava NullPointerException nella registrazione del log APP in caso di response null
- * CORE Corretto bug che causava NullPointerException in caso di credenziale operatore senza tag <rep>
- * CORE Aggiunta in RestTemplate la gestione dei retry
 - Nuova proprietà spl.client-rest.retry.uri-matchers con valore default /**
 - Nuova proprietà spl.client-rest.retry.excluded-uri-matchers
 - Nuova proprietà spl.client-rest.retry.enabled con valore default false
 - Nuova proprietà spl.client-rest.retry.max-retries con valore default 3
 - Nuova proprietà spl.client-rest.retry.with-timeout con valore default 0
 - Nuova proprietà spl.client-rest.retry.fixed-backoff con valore default 0
- * OPENAPI springdoc-openapi-starter 2.4.0
- * AUDITLOG logback 1.5.3
- * AUDITLOG slf4j 2.0.12

** 2024.03.01

- * ALL spring boot 3.2.3
- * CORE Impostato valore di default alla property `server.tomcat.max-http-form-post-size=2MB`
- * CORE Modificata definizione del controller /logout al fine di evitare che appaia nello swagger

** 2024.02.01

- * ALL spring boot 3.2.2
- * BATCH Aggiunta modulo springline2-batch

** 2024.01.01

- * ALL spring boot 3.2.1
- * CORE spring cloud 2023.0.0
- * CORE Gestione exception NoResourceFoundException per ritorno status code 404.

** 2023.06.05

- * CORE Corretto bug che causava un troncamento della response in caso di esclusione di mimetype

** 2023.06.04

- * CORE Gestione exception MissingServletRequestPartException per ritorno status code 400.
- * CORE modificato valore di default della proprietà spl.http.logging.app.excluded-content-types:application/x-www-form-urlencoded,multipart/form-data
- * CORE Corretto bug che causava NullPointerException durante l'uso del RestTemplate in caso di content type null
- * CORE Modificata la gestione della esclusione del log APP in funzione del content type.

** 2023.06.03

- * CORE Corretto bug che, in caso di autenticazione API MANAGER con chiavi SANDBOX, impostava il circuito REALE invece che VIRTUALE
- * CORE Modificata la sequenza dei filter che gestiscono l'autenticazione per gestire il caso in cui un endpoint è esposto sia in IPDC che SSO

** 2023.06.02

- * CORE Gli header HTTP senza valore sono ora considerati assenti
- * CORE Corretto bug che causava NullPointerException in caso di credenziale operatore senza L1 o L2 o L3 (autenticazione API Manager)

```

** 2023.06.01
* ALL spring boot 3.1.12
* CORE Aggiunto un custom StrictHttpFirewall che converte in UTF-8 i valori degli
  header http
  - Nuova proprietà spl.security.http-firewall.enabled con valore default false
  - Nuova proprietà spl.security.http-firewall.regex con valore default
    [\\p{IsAssigned}&&[^\p{IsControl}]]\\t]*
* CORE Se viene impostato a DEBUG il log della classe
  it.ariaspa.springline2.filter.TracerFilter vengono loggate informazioni della
  request
* CORE Aggiunta variabile (traceId) MDC e valorizzata con i valori di trace
* CORE Modificato pattern layout del log in console e file per stampare la variabile
  MDC traceId
* OPENAPI springdoc-openapi-starter 2.5.0

** 2023.05.01
* ALL spring boot 3.1.10
* CORE jjwt 0.12.5
* CORE Migliorata la risposta in caso di errore di validazione dell'input
* CORE Corretto bug che causava NullPointerException nella registrazione del log APP
  in caso di response null
* CORE Corretto bug che causava NullPointerException in caso di credenziale operatore
  senza tag <rep>
* CORE Aggiunta in RestTemplate la gestione dei retry
  - Nuova proprietà spl.client-rest.retry.uri-matchers con valore default /**
  - Nuova proprietà spl.client-rest.retry.excluded-uri-matchers
  - Nuova proprietà spl.client-rest.retry.enabled con valore default false
  - Nuova proprietà spl.client-rest.retry.max-retries con valore default 3
  - Nuova proprietà spl.client-rest.retry.with-timeout con valore default 0
  - Nuova proprietà spl.client-rest.retry.fixed-backoff con valore default 0
* OPENAPI springdoc-openapi-starter 2.4.0
* AUDITLOG logback 1.5.3
* AUDITLOG slf4j 2.0.12

** 2023.04.01
* ALL spring boot 3.1.9
* CORE Impostato valore di default alla property
  server.tomcat.max-http-form-post-size=2MB
* CORE Modificata definizione del controller /logout al fine di evitare che appaia
  nello swagger

** 2023.03.01
* ALL spring boot 3.1.8

** 2023.02.01
* ALL spring boot 3.1.7
* CORE Aggiunta la gestione del JWT in cookie
  - Nuova proprietà spl.security.authentication.jwt.header.enabled con valore
    default true per abilitare la ricerca del jwt nell'header.
  - Nuova proprietà spl.security.authentication.jwt.cookie.enabled con valore
    default false per abilitare la ricerca del jwt nel cookie.
  - Nuova proprietà spl.security.authentication.jwt.cookie.name con valore di
    default springline2Token per personalizzare il nome del cookie.
  - Classe JwtToken nuovo metodo generateCookie(claims) per generare un cookie
    contenente il jwt con i claims specificati.
  - Classe JwtToken nuovo metodo generateCookie() per generare un cookie
    contenente il jwt senza claims.
* CORE Spostata la chiamata a getCanonicalHostName per ottenere il nome macchina allo
  start dell'applicazione.
* OPENAPI springdoc-openapi-starter 2.3.0
* JMS Bug-fix nella definizione degli aspect join point
* JMS Spostata la chiamata a getCanonicalHostName per ottenere il nome macchina allo
  start dell'applicazione.
* WS Spostata la chiamata a getCanonicalHostName per ottenere il nome macchina allo
  start dell'applicazione.
* AUDITLOG logback 1.4.14

** 2023.01.01
* Prima versione

```

16. Appendice E – Configurazioni logback¹¹

16.1 logback-springline2.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property name="CONSOLE_LOG_PATTERN"
    value="${CONSOLE_LOG_PATTERN:-%clr(%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint}
%clr(${LOG_LEVEL_PATTERN:-%5p}){ } %clr(${PID:-}){magenta} %clr(---

```

¹¹ Le configurazioni riportate si riferiscono alla versione più recente di SPRINGLINE2

```

%esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP}[%15.15t] ${LOG_CORRELATION_PATTERN:-}{faint}%clr(%-
40.40logger{39}){cyan} %clr(%X{traceId}){yellow}%clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <property name="FILE_LOG_PATTERN"
        value="${FILE_LOG_PATTERN:-%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}
${LOG_LEVEL_PATTERN:-%5p} ${PID:-} --- %esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP}[%t]
${LOG_CORRELATION_PATTERN:-}%-40.40logger{39} %X{traceId}: %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <include
        resource="org/springframework/boot/logging/logback/defaults.xml" />
    <property name="LOG_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/application.log" />
    <property name="MON_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/MON.log" />
    <property name="APP_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/APP.log" />
    <property name="AUDIT_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/AUDIT.log" />
    <include
        resource="org/springframework/boot/logging/logback/console-appender.xml" />
    <include
        resource="org/springframework/boot/logging/logback/file-appender.xml" />
    <appender name="MON" class="ch.qos.logback.core.FileAppender">
        <file>${MON_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%msg%n</pattern>
        </encoder>
    </appender>
    <appender name="APP" class="ch.qos.logback.core.FileAppender">
        <file>${APP_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%msg%n</pattern>
        </encoder>
    </appender>
    <appender name="AUDIT" class="ch.qos.logback.core.FileAppender">
        <file>${AUDIT_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%msg%n</pattern>
        </encoder>
    </appender>
    <logger name="MON" level="ALL" additivity="false">
        <appender-ref ref="MON" />
    </logger>
    <logger name="APP" level="ALL" additivity="false">
        <appender-ref ref="APP" />
    </logger>
    <logger name="AUDIT" level="ALL" additivity="false">
        <appender-ref ref="AUDIT" />
    </logger>
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
        <appender-ref ref="FILE" />
    </root>
</configuration>

```

16.2 logback-springline2-ocp.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <property name="CONSOLE_LOG_PATTERN"
        value="${CONSOLE_LOG_PATTERN:-%clr(%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint}
%clr(${LOG_LEVEL_PATTERN:-%5p}){} %clr(${PID:-}){magenta} %clr(---
%esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP}[%15.15t] ${LOG_CORRELATION_PATTERN:-}{faint}%clr(%-
40.40logger{39}){cyan} %clr(%X{traceId}){yellow}%clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <property name="FILE_LOG_PATTERN"
        value="${FILE_LOG_PATTERN:-%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}
${LOG_LEVEL_PATTERN:-%5p} ${PID:-} --- %esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP}[%t]
${LOG_CORRELATION_PATTERN:-}%-40.40logger{39} %X{traceId}: %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <include
        resource="org/springframework/boot/logging/logback/defaults.xml" />
    <include
        resource="org/springframework/boot/logging/logback/console-appender.xml" />
    <appender name="MON" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>MON->%msg%n</pattern>

```

```

        </encoder>
    </appender>
    <appender name="APP" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>APP->%msg%n</pattern>
        </encoder>
    </appender>
    <appender name="AUDIT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>AUDIT->%msg%n</pattern>
        </encoder>
    </appender>
    <logger name="MON" level="ALL" additivity="false">
        <appender-ref ref="MON" />
    </logger>
    <logger name="APP" level="ALL" additivity="false">
        <appender-ref ref="APP" />
    </logger>
    <logger name="AUDIT" level="ALL" additivity="false">
        <appender-ref ref="AUDIT" />
    </logger>
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
    </root>
</configuration>

```

16.3 logback-springline2-encrypt.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <conversionRule conversionWord="encryptmsg"
    converterClass="it.ariaspa.springline2.logging.logback.EncryptMsgConverter" />

    <springProperty name="ksLocation" source="spl.logging.encrypt.keyStore.location"/>
    <springProperty name="ksPassword" source="spl.logging.encrypt.keyStore.password"/>
    <springProperty name="ksType" source="spl.logging.encrypt.keyStore.type"/>
    <springProperty name="kAlias" source="spl.logging.encrypt.keystore.alias"/>

    <property name="CONSOLE_LOG_PATTERN"
        value="${CONSOLE_LOG_PATTERN:-%clr(%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint}
%clr(%d{${LOG_LEVEL_PATTERN:-%5p}}){} %clr(%d{${PID:-}}){magenta} %clr(---
%esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP} [%15.15t] %d{${LOG_CORRELATION_PATTERN:-}}){faint}%clr(%-
40.40logger{39}){cyan} %clr(%X{traceId}){yellow}%clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <property name="FILE_LOG_PATTERN"
        value="${FILE_LOG_PATTERN:-%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}
${LOG_LEVEL_PATTERN:-%5p} ${PID:-} --- %esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP} [%t]
${LOG_CORRELATION_PATTERN:-}%-40.40logger{39} %X{traceId}: %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx}}"/>
    <include
        resource="org/springframework/boot/logging/logback/defaults.xml" />
    <property name="LOG_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/application.log" />
    <property name="MON_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/MON.log" />
    <property name="APP_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/APP.log" />
    <property name="AUDIT_FILE"
        value="${LOG_PATH:-${LOG_TEMP:-${java.io.tmpdir:-/tmp}}}/AUDIT.log" />
    <include
        resource="org/springframework/boot/logging/logback/console-appender.xml" />
    <include
        resource="org/springframework/boot/logging/logback/file-appender.xml" />
    <appender name="MON" class="ch.qos.logback.core.FileAppender">
        <file>${MON_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%encryptmsg${ksLocation},${ksPassword},${ksType},${kAlias}%n</pattern>
        </encoder>
    </appender>
    <appender name="APP" class="ch.qos.logback.core.FileAppender">
        <file>${APP_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%encryptmsg${ksLocation},${ksPassword},${ksType},${kAlias}%n</pattern>
        </encoder>
    </appender>
    <appender name="AUDIT" class="ch.qos.logback.core.FileAppender">

```

```

        <file>${AUDIT_FILE}</file>
        <append>true</append>
        <encoder>
            <pattern>%encryptmsg(${ksLocation},${ksPassword},${ksType},${kAlias})%n</pattern>
        </encoder>
    </appender>
    <logger name="MON" level="ALL" additivity="false">
        <appender-ref ref="MON" />
    </logger>
    <logger name="APP" level="ALL" additivity="false">
        <appender-ref ref="APP" />
    </logger>
    <logger name="AUDIT" level="ALL" additivity="false">
        <appender-ref ref="AUDIT" />
    </logger>
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
        <appender-ref ref="FILE" />
    </root>
</configuration>

```

16.4 logback-springline2-ocp-encrypt.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <conversionRule conversionWord="encryptmsg"
        converterClass="it.ariaspa.springline2.logging.logback.EncryptMsgConverter" />

    <springProperty name="ksLocation" source="logging.encrypt.keyStore.location"/>
    <springProperty name="ksPassword" source="logging.encrypt.keyStore.password"/>
    <springProperty name="ksType" source="logging.encrypt.keyStore.type"/>
    <springProperty name="kAlias" source="logging.encrypt.keystore.alias"/>

    <property name="CONSOLE_LOG_PATTERN"
        value="${CONSOLE_LOG_PATTERN:-%clr(%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}){faint}
%clr(${LOG_LEVEL_PATTERN:-%5p}){ } %clr(${PID:-}){magenta} %clr(---
%esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP}[%15.15t] ${LOG_CORRELATION_PATTERN:-}){faint}%clr(%-
40.40logger{39}){cyan} %clr(%X{traceId}){yellow}%clr(:){faint} %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx} }"/>
    <property name="FILE_LOG_PATTERN"
        value="${FILE_LOG_PATTERN:-%d{${LOG_DATEFORMAT_PATTERN:-yyyy-MM-dd'T'HH:mm:ss.SSSXXX}}
${LOG_LEVEL_PATTERN:-%5p} ${PID:-} --- %esb() {APPLICATION_NAME}%esb{APPLICATION_GROUP} [%t]
${LOG_CORRELATION_PATTERN:-}%-40.40logger{39} %X{traceId}: %m%n${LOG_EXCEPTION_CONVERSION_WORD:-%wEx} }"/>
    <include
        resource="org/springframework/boot/logging/logback/defaults.xml" />
    <include
        resource="org/springframework/boot/logging/logback/console-appender.xml" />
    <appender name="MON" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>MON->%encryptmsg(${ksLocation},${ksPassword},${ksType},${kAlias})%n</pattern>
        </encoder>
    </appender>
    <appender name="APP" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>APP->%encryptmsg(${ksLocation},${ksPassword},${ksType},${kAlias})%n</pattern>
        </encoder>
    </appender>
    <appender name="AUDIT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>AUDIT->%encryptmsg(${ksLocation},${ksPassword},${ksType},${kAlias})%n</pattern>
        </encoder>
    </appender>
    <logger name="MON" level="ALL" additivity="false">
        <appender-ref ref="MON" />
    </logger>
    <logger name="APP" level="ALL" additivity="false">
        <appender-ref ref="APP" />
    </logger>
    <logger name="AUDIT" level="ALL" additivity="false">
        <appender-ref ref="AUDIT" />
    </logger>
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
    </root>
</configuration>

```